

iska bhi solution do

Below is the complete solution of the **University of the Punjab – BS 4 Years – Mobile Application Development (EI-333) Spring 2025** paper.

Q1. Short Questions ($6 \times 5 = 30$)

1. Explain the function of R.java in Android applications. How is it generated and used?

Answer

R.java is an automatically generated Java class that contains unique IDs for all application resources.

These resources include:

- Layouts
- Strings
- Images (Drawables)
- Colors
- Menus
- IDs

How is it generated?

- Android Studio automatically generates the **R** class during project compilation.
- It is regenerated whenever a resource is added, deleted, or modified in the **res** folder.

Uses

- Access layouts
- Access strings
- Access images
- Access buttons and views

Example

```
setContentView(R.layout.activity_main);  
  
TextView txt = findViewById(R.id.textView);  
  
String name = getString(R.string.app_name);
```

Advantages

- Avoids hardcoding resource IDs.
 - Makes coding easier.
 - Reduces errors.
 - Improves code readability.
-

2. What are Event Handlers in Android? Illustrate with an example of button click.

Answer

An **Event Handler** is code that responds when a user performs an action such as:

- Button Click
- Long Click
- Touch
- Swipe
- Key Press

The most common event is **Button Click** using **OnClickListener**.

Example

```
Button btn = findViewById(R.id.btnSave);

btn.setOnClickListener(new View.OnClickListener() {

    @Override
    public void onClick(View view) {

        Toast.makeText(MainActivity.this,
            "Button Clicked",
            Toast.LENGTH_SHORT).show();

    }

});
```

Common Event Listeners

- OnClickListener
- OnLongClickListener
- onTouchListener
- OnKeyListener

- OnFocusChangeListener
-

3. Differentiate between Explicit and Implicit Intents. Provide use-case examples.

Explicit Intent

An Explicit Intent specifies the exact Activity to open.

Example

```
Intent intent = new Intent(MainActivity.this,
    SecondActivity.class);

startActivity(intent);
```

Use Cases

- Login → Home
 - Home → Profile
 - Splash → Main Activity
-

Implicit Intent

An Implicit Intent requests an action without specifying the target Activity.

Example

```
Intent intent = new Intent(Intent.ACTION_VIEW);

intent.setData(Uri.parse("https://google.com"));

startActivity(intent);
```

Use Cases

- Open Browser
 - Make Phone Call
 - Send Email
 - Open Camera
-

- Share Content

Difference Table

Explicit Intent	Implicit Intent
Opens a specific Activity	Performs an action
App knows destination	Android selects suitable app
Used inside own app	Used with other apps

4. What are Android Widgets? Describe any two commonly used widgets.

Answer

Android **Widgets** are UI components used to build user interfaces.

Examples:

- Button
 - TextView
 - EditText
 - ImageView
 - RecyclerView
 - CheckBox
-

1. Button

Used to perform an action when clicked.

Example

```
<Button
android:id="@+id/btnLogin"
android:text="Login"/>
btnLogin.setOnClickListener(v -> {

});
```

2. EditText

Used to receive input from the user.

Example

```
<EditText
android:id="@+id/editName"
android:hint="Enter Name"/>
String name = editName.getText().toString();
```

5. Explain the steps to create and use an SQLite database in Android.

Answer

SQLite is a lightweight relational database used to store data locally.

Steps

Step 1

Create a helper class

```
public class DBHelper extends SQLiteOpenHelper
```

Step 2

Create database

```
SQLiteDatabase db =
getWritableDatabase();
```

Step 3

Create Table

```
db.execSQL(
"CREATE TABLE Student(id INTEGER PRIMARY KEY,name TEXT)"
);
```

Step 4

Insert Data

```
ContentValues values = new ContentValues();  
values.put("name", "Ali");  
db.insert("Student", null, values);
```

Step 5

Retrieve Data

```
Cursor cursor = db.rawQuery(  
"SELECT * FROM Student", null);
```

Step 6

Update/Delete Data

```
db.update(...);  
db.delete(...);
```

Advantages

- Offline storage
 - Fast
 - Lightweight
 - No Internet required
-

6. What is Android Native Development Kit (NDK)? Why and when should it be used?

Answer

The **Android Native Development Kit (NDK)** is a set of tools that allows developers to write parts of Android apps using **C** and **C++**.

Why use NDK?

- Better performance
- Code reuse
- Access native libraries
- Game development
- Image processing

When should it be used?

- 3D Games
- Machine Learning
- Audio Processing
- Video Editing
- Heavy Computation

Advantages

- Faster execution
 - Better performance
 - Supports existing C/C++ libraries
-

Q2 (1)

Create an Android app that retrieves a list of posts from a Web API and displays them using RecyclerView. Explain the flow.

Answer

Architecture

```
API
  ↓
Retrofit
  ↓
Repository
  ↓
ViewModel
  ↓
RecyclerView Adapter
  ↓
RecyclerView
```

Step 1

Add Internet Permission

```
<uses-permission  
android:name="android.permission.INTERNET"/>
```

Step 2

Create Data Model

```
public class Post{  
  
    int id;  
  
    String title;  
  
    String body;  
  
}
```

Step 3

Create API Interface

```
@GET("posts")  
  
Call<List<Post>> getPosts();
```

Step 4

Use Retrofit

```
Retrofit retrofit =  
new Retrofit.Builder()  
    .baseUrl("https://jsonplaceholder.typicode.com/")  
    .build();
```

Step 5

Display Data

```
recyclerView.setAdapter(adapter);
```

Flow

```
User Opens App
      ↓
Retrofit calls API
      ↓
JSON Response
      ↓
Convert into Objects
      ↓
RecyclerView Adapter
      ↓
Display Posts
```

Q2 (2)

Design an app with two Activities. First Activity takes user input, second Activity displays it.

Answer

Activity 1

```
EditText editName;

Button btn;

btn.setOnClickListener(v->{

    Intent intent =
    new Intent(MainActivity.this,
    SecondActivity.class);

    intent.putExtra("name",
    editName.getText().toString());

    startActivity(intent);

});
```

Activity 2

```
TextView txt;

String name =
```

```
getIntent().getStringExtra("name");  
  
txt.setText(name);
```

Intent Flow

```
MainActivity  
    ↓  
Intent  
    ↓  
putExtra()  
    ↓  
SecondActivity  
    ↓  
getStringExtra()  
    ↓  
Display Data
```

Activity Transition

```
startActivity(intent);
```

Back button returns to MainActivity.

Q2 (3)

Develop an Android app using NotificationManager to display notifications at specific intervals. Include dynamic notification content and action handling.

Answer

Step 1

Create Notification Channel

```
NotificationChannel channel =  
    new NotificationChannel(  
        "1",  
        "Reminder",  
        NotificationManager.IMPORTANCE_HIGH);
```

Step 2

Create Notification

```
NotificationCompat.Builder builder =  
new NotificationCompat.Builder(this, "1")  
  
.setSmallIcon(R.drawable.ic_launcher)  
  
.setContentTitle("Reminder")  
  
.setContentText("Study Android Today");
```

Step 3

Show Notification

```
NotificationManagerCompat.from(this)  
  
.notify(1, builder.build());
```

Step 4

Schedule Notification

Use:

- AlarmManager
 - WorkManager
-

Dynamic Notification

```
builder.setContentText(  
"Hello " + username);
```

Action Button

```
builder.addAction(  
R.drawable.ic_open,  
"Open",  
pendingIntent);
```

Flow

```
AlarmManager
  ↓
BroadcastReceiver
  ↓
NotificationManager
  ↓
Notification Appears
  ↓
User Clicks Action
  ↓
Open Activity
```

Exam Tips for 10-Mark Questions

For every long question, follow this structure to maximize marks:

1. Definition
2. Purpose
3. Architecture/Flow Diagram
4. Code Example
5. Explanation of Code
6. Advantages
7. Conclusion

This format is ideal for Punjab University exams and helps you secure higher marks by covering both theory and practical aspects.

Mobile Computing Domain (Complete Notes for BSIT – Mobile Application Development EI-333)

This is the **first and most important chapter** of your syllabus. Questions from this chapter are frequently asked in Punjab University exams.

1. What is Mobile Computing?

Definition

Mobile Computing is a technology that enables users to access data, applications, and network services anytime and anywhere using portable devices connected through wireless communication.

In simple words:

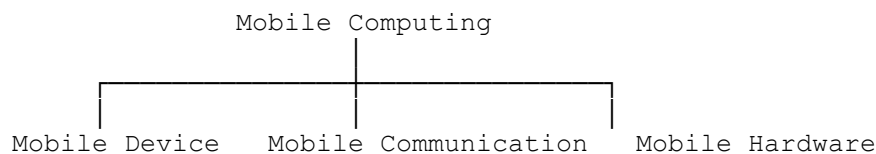
Mobile computing allows people to communicate and use applications while moving from one place to another without requiring a wired connection.

Example

- Using WhatsApp while traveling
- Watching YouTube on a mobile phone
- Online banking through a smartphone
- Using Google Maps while driving
- Food delivery apps (Foodpanda, Uber Eats)

2. Components of Mobile Computing

Mobile Computing consists of **three main components**.



A. Mobile Device

A mobile device is a portable electronic device used by users.

Examples

- Smartphone
- Tablet
- Laptop
- Smartwatch
- PDA (Personal Digital Assistant)

Features

- Portable

- Battery-powered
 - Internet connectivity
 - Touchscreen
 - Sensors (GPS, Camera, Accelerometer)
-

B. Mobile Communication

Mobile communication refers to the wireless transmission of data between mobile devices and networks.

Technologies

- GSM
- CDMA
- Wi-Fi
- Bluetooth
- NFC
- 3G
- 4G LTE
- 5G

Purpose

- Voice calls
 - Video calls
 - Internet browsing
 - Messaging
 - File sharing
-

C. Mobile Hardware

Hardware includes all physical components of a mobile device.

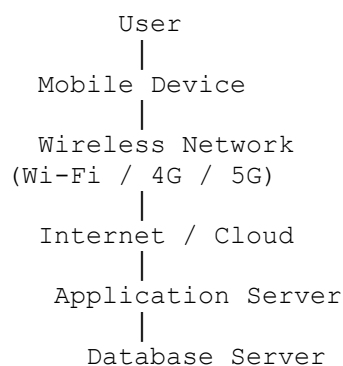
Examples

- Processor (CPU)
 - RAM
 - Storage
 - Battery
 - Display
 - Camera
-

- GPS
- Speaker
- Microphone
- SIM card

3. Mobile Computing Architecture

The architecture of mobile computing shows how users communicate with servers through wireless networks.



Explanation

Step 1

The user sends a request using a smartphone.

Example:

Open Facebook.

↓

Step 2

The request goes through a wireless network.

↓

Step 3

The request reaches the Internet.

↓

Step 4

The application server processes the request.

↓

Step 5

The server retrieves data from the database.

↓

Step 6

The response is sent back to the mobile device.

4. Characteristics of Mobile Computing

These are commonly asked in short questions.

1. Portability

Users can carry the device anywhere.

Example:

Using a smartphone while traveling.

2. Mobility

Users can access information while moving.

Example:

Using Google Maps during driving.

3. Connectivity

Continuous wireless connection.

Examples

- Wi-Fi
 - 4G
 - 5G
-

4. Interactivity

Users can communicate instantly.

Examples

- WhatsApp
 - Messenger
 - Zoom
-

5. Personalization

Each user has personalized settings.

Examples

- Wallpapers
 - Language
 - App preferences
-

6. Convenience

Access services anytime and anywhere.

Examples

- Online shopping
 - Online banking
-

7. Location Awareness

Applications can determine the user's location using GPS.

Examples

- Google Maps
- Uber
- Careem

8. Scalability

Supports millions of users simultaneously.

Example:

Instagram serves millions of users worldwide.

5. Advantages of Mobile Computing

These are important for both short and long questions.

1. Anywhere Access

Users can access information from any location.

2. Better Communication

Supports instant messaging and video calls.

3. Increased Productivity

Employees can work remotely.

4. Easy Data Sharing

Share files quickly using:

- Bluetooth
- Wi-Fi
- Internet

5. Time Saving

Tasks can be completed quickly without visiting physical locations.

Example:

Online banking.

6. Entertainment

Users can watch movies, play games, and stream music.

7. Online Education

Attend classes using:

- Zoom
- Google Meet
- Microsoft Teams

8. Navigation

Provides GPS-based navigation.

Example:

Google Maps.

6. Disadvantages of Mobile Computing

1. Security Risks

Devices can be hacked or infected with malware.

2. Privacy Issues

Personal information may be stolen.

3. Battery Limitations

Battery drains quickly during heavy usage.

4. Network Dependency

Internet connection is required for many services.

5. Small Screen Size

Limited display compared to desktop computers.

6. Data Loss

Loss or theft of a device may result in loss of important data.

7. Performance Limitations

Mobile devices generally have lower processing power than desktop computers.

7. Applications of Mobile Computing

This question appears frequently.

Banking

Examples:

- Mobile banking
 - Online transactions
-

Healthcare

Examples:

- Telemedicine
 - Online appointments
 - Electronic Medical Records (EMR)
-

Education

Examples:

- LMS
 - Google Classroom
 - Zoom
-

Transportation

Examples:

- Uber
 - Careem
 - Google Maps
-

Business

Examples:

- Remote work
 - Email
 - Cloud storage
-

Entertainment

Examples:

- YouTube
 - Netflix
 - Spotify
-

E-Commerce

Examples:

- Amazon
 - Daraz
 - eBay
-

Social Media

Examples:

- Facebook
 - Instagram
 - TikTok
 - X (Twitter)
-

Gaming

Examples:

- PUBG
- Free Fire
- Clash of Clans

Smart Homes

Examples:

- Smart lights
- Smart cameras
- Smart locks

8. Types of Mobile Networks

Technology	Speed	Coverage	Example
Bluetooth	Low	10–100 m	Wireless earphones
Wi-Fi	High	Local Area	Home internet
3G	Medium	Wide Area	Mobile Internet
4G LTE	High	Wide Area	HD Streaming
5G	Very High	Wide Area	IoT, Smart Cities
NFC	Very Short	Few centimeters	Contactless payments

9. Mobile Computing vs Traditional Computing

Mobile Computing	Traditional Computing
Portable	Not portable
Wireless connectivity	Mostly wired
Battery powered	Continuous power supply

Mobile Computing	Traditional Computing
-------------------------	------------------------------

Can be used anywhere	Usually fixed location
----------------------	------------------------

Small screen	Large screen
--------------	--------------

Limited hardware	More powerful hardware
------------------	------------------------

10. Real-Life Examples

- Using WhatsApp while traveling.
 - Booking a ride through Uber or Careem.
 - Paying bills using a banking app.
 - Shopping through Daraz or Amazon.
 - Tracking delivery using Foodpanda.
 - Attending online classes on Zoom.
 - Navigating with Google Maps.
 - Streaming videos on YouTube.
-

Important Exam Questions

Short Questions (2–5 Marks)

1. Define Mobile Computing.
2. What are the components of Mobile Computing?
3. List the characteristics of Mobile Computing.
4. Write four advantages of Mobile Computing.
5. Write four disadvantages of Mobile Computing.
6. Mention any five applications of Mobile Computing.

Long Questions (10–15 Marks)

1. Explain Mobile Computing with a neat architecture diagram.
 2. Discuss the characteristics, advantages, disadvantages, and applications of Mobile Computing.
 3. Compare Mobile Computing with Traditional Computing.
-

MCQs for Practice

1. Mobile Computing mainly depends on:

- A) Wired Network
- B) Wireless Communication ✓
- C) USB Connection
- D) Optical Fiber

2. Which of the following is NOT a mobile device?

- A) Smartphone
- B) Tablet
- C) Desktop Computer ✓
- D) Smartwatch

3. GPS is mainly used for:

- A) Gaming
- B) Location Tracking ✓
- C) Printing
- D) Email

4. Which technology provides the fastest mobile internet among these?

- A) 3G
- B) 4G
- C) 5G ✓
- D) Bluetooth

5. Which is NOT an advantage of Mobile Computing?

- A) Portability
- B) Mobility
- C) Requires wired connection ✓
- D) Easy communication

Quick Revision (5 Minutes Before Exam)

- **Definition:** Technology that allows access to data and services anytime, anywhere using wireless mobile devices.
- **Components:** Mobile Device, Mobile Communication, Mobile Hardware.
- **Characteristics:** Portability, Mobility, Connectivity, Interactivity, Personalization, Convenience, Location Awareness, Scalability.
- **Advantages:** Anywhere access, Better communication, Productivity, Data sharing, Time saving.
- **Disadvantages:** Security, Privacy, Battery, Network dependency, Small screen.

- **Applications:** Banking, Healthcare, Education, Transportation, Business, Entertainment, E-commerce, Social Media, Gaming, Smart Homes.

These notes are comprehensive and exam-oriented, suitable for Punjab University's EI-333 syllabus.

Event Handling in Android (Complete Notes for BSIT – Mobile Application Development EI-333)

Event Handling is one of the most important topics in Android Development. Every Android application responds to user actions such as button clicks, touch events, typing text, or selecting menu items. This topic is frequently asked in **Punjab University exams** for 5, 10, and 15 marks.

What is Event Handling?

Definition

Event Handling is the process of detecting and responding to user actions (events) in an Android application.

In simple words:

Event Handling allows an application to perform an action when the user interacts with a widget.

What is an Event?

An **Event** is an action performed by the user or the Android system.

Examples of Events

- Button Click
- Long Press
- Screen Touch
- Typing in EditText

- Checkbox Selection
 - Spinner Selection
 - Menu Click
 - Swipe
 - Device Rotation
-

What is an Event Handler?

An **Event Handler** is a piece of code that executes when an event occurs.

For example:

User Clicks Login Button

↓

Button Click Event Occurs

↓

OnClickListener Executes

↓

Login Screen Opens

Event Handling Architecture

User Action

↓

Event Generated

↓

Listener Detects Event

↓

Callback Method Executes

↓

Application Responds

Components of Event Handling

There are three main parts:

Event Handling

|

|— Event Source

|— Event Listener

|— Event Handler (Callback Method)

1. Event Source

The widget that generates the event.

Examples:

- Button
- EditText
- CheckBox
- RadioButton
- Spinner
- ImageView

Example:

Button

↓

Click Event Generated

2. Event Listener

The object that listens for the event.

Example:

```
button.setOnClickListener(...)
```

3. Callback Method

The method that is automatically called when the event occurs.

Example:

```
public void onClick(View view) {  
  
}
```

Types of Event Listeners

Android provides many listeners.

Listener	Purpose
OnClickListener	Button Click
OnLongClickListener	Long Press
OnTouchListener	Touch Screen
OnFocusChangeListener	Focus Changed
OnKeyListener	Keyboard Key
OnCheckedChangeListener	CheckBox / Switch
OnItemClickListener	ListView Item
OnItemSelectedListener	Spinner Item
TextWatcher	Text Changed

1. OnClickListener ★ ★ ★ ★ ★

This is the most commonly used listener.

Definition

It is triggered when the user clicks a View.

Syntax

```
button.setOnClickListener(new View.OnClickListener() {
```

```
        @Override
        public void onClick(View view) {

        }

    });
```

Complete Example

XML

```
<Button
    android:id="@+id/btnLogin"
    android:text="Login"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>
```

Java

```
Button btnLogin;

btnLogin = findViewById(R.id.btnLogin);

btnLogin.setOnClickListener(new View.OnClickListener() {

    @Override
    public void onClick(View view) {

        Toast.makeText(MainActivity.this,
            "Login Successful",
            Toast.LENGTH_SHORT).show();

    }

});
```

Output

User Clicks Login

↓

Toast Appears

"Login Successful"

2. OnLongClickListener

Definition

Called when the user presses and holds a view.

Example

```
button.setOnLongClickListener(new View.OnLongClickListener() {  
    @Override  
    public boolean onLongClick(View view) {  
        Toast.makeText(MainActivity.this,  
            "Long Click",  
            Toast.LENGTH_SHORT).show();  
        return true;  
    }  
});
```

Uses

- Delete item
 - Show options
 - Copy text
-

3. onTouchListener

Definition

Handles touch events.

Examples

- Finger Touch
- Swipe

- Drag
 - Multi-touch
-

Example

```
button.setOnTouchListener(new View.OnTouchListener() {  
  
    @Override  
  
    public boolean onTouch(View view,  
                           MotionEvent event) {  
  
        return true;  
  
    }  
  
});
```

MotionEvent Actions

Action	Meaning
ACTION_DOWN	Finger touches screen
ACTION_UP	Finger removed
ACTION_MOVE	Finger moving

4. OnFocusChangeListener

Definition

Called when a view gains or loses focus.

Example

```
editText.setOnFocusChangeListener(  
  
(view,
```

Fragments in Android (Complete Notes for BSIT – Mobile Application Development EI-333)

Fragments are one of the **most important topics** in Android Development. They are frequently asked in **Punjab University** exams for **5, 10, and 15 marks**. Fragments help developers create flexible and reusable user interfaces.

What is a Fragment?

Definition

A **Fragment** is a reusable portion of an Activity's user interface that has its own layout, lifecycle, and behavior.

In simple words:

A Fragment is a small part (or sub-screen) of an Activity that can be added, removed, or replaced dynamically.

Unlike an Activity, a Fragment **cannot exist independently**. It must always be hosted inside an Activity.

Real-Life Example

Think of a **newspaper**.

- The whole newspaper = **Activity**
- Each section (Sports, Business, Entertainment) = **Fragment**

Activity	
Fragment 1 (Menu)	Fragment 2 (Content)

Why Do We Use Fragments?

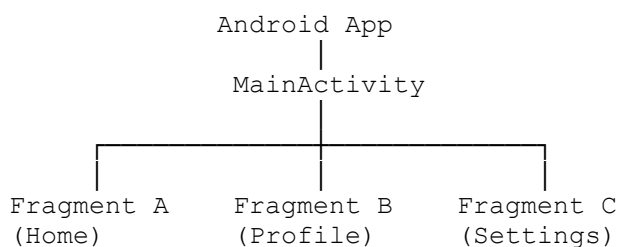
Fragments are used because they:

- Make UI reusable.
 - Support different screen sizes (mobile/tablet).
 - Allow dynamic UI changes.
 - Improve modularity.
 - Simplify maintenance.
-

Features of Fragments

- Reusable UI component
 - Has its own lifecycle
 - Has its own layout
 - Can receive user input
 - Can communicate with Activities
 - Can be added, removed, or replaced at runtime
-

Fragment Architecture



Fragment vs Activity

Fragment	Activity
Part of an Activity	Independent screen
Cannot exist alone	Can exist independently
Reusable	Less reusable

Fragment	Activity
Has its own lifecycle	Has its own lifecycle
Lightweight	Heavier
Used inside Activity	Represents a complete screen

Types of Fragments

There are mainly four types:

1. Static Fragment
 2. Dynamic Fragment
 3. Dialog Fragment
 4. List Fragment
-

1. Static Fragment

A Static Fragment is declared directly in the Activity's XML layout.

Example

```
<fragment
    android:id="@+id/homeFragment"
    android:name="com.example.HomeFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

Advantages

- Easy to implement.
- Suitable for fixed layouts.

Disadvantages

- Cannot change during runtime.
-

2. Dynamic Fragment

A Dynamic Fragment is added, replaced, or removed using Java/Kotlin code.

Example

```
FragmentManager fm = getSupportFragmentManager();  
  
FragmentTransaction ft = fm.beginTransaction();  
  
ft.replace(R.id.fragmentContainer, new HomeFragment());  
  
ft.commit();
```

Advantages

- Flexible
- Runtime changes
- Used in modern apps

3. Dialog Fragment

Used to display dialogs.

Example:

```
Delete Student?  
  
[ YES ]    [ NO ]
```

4. List Fragment

Displays a list of items.

Example:

```
Students  
  
Ali  
  
Ahmed  
  
Sara  
  
Bilal
```

Creating a Fragment

Step 1

Create a Fragment class.

```
public class HomeFragment extends Fragment {  
  
}
```

Step 2

Create XML layout.

fragment_home.xml

```
<TextView  
  
    android:text="Home Fragment"  
  
    android:layout_width="wrap_content"  
  
    android:layout_height="wrap_content"/>
```

Step 3

Inflate Layout

```
@Override  
  
public View onCreateView(  
  
    LayoutInflater inflater,  
  
    ViewGroup container,  
  
    Bundle savedInstanceState) {  
  
    return inflater.inflate(  
  
        R.layout.fragment_home,  
  
        container,  
  
        false);  
  
}
```

Step 4

Display Fragment

```
FragmentManager manager=
getSupportFragmentManager();
FragmentManager transaction=
manager.beginTransaction();
transaction.replace(
R.id.fragmentContainer,
new HomeFragment());
transaction.commit();
```

Fragment Lifecycle

Like Activities, Fragments also have a lifecycle.

```
onAttach()
↓
onCreate()
↓
onCreateView()
↓
onViewCreated()
↓
onStart()
↓
onResume()
↓
(App Running)
```

↓
onPause()
↓
onStop()
↓
onDestroyView()
↓
onDestroy()
↓
onDetach()

Fragment Lifecycle Methods

1. onAttach()

Called when the Fragment is attached to the Activity.

```
@Override  
public void onAttach(Context context) {  
    super.onAttach(context);  
}
```

2. onCreate()

Initialize variables.

```
@Override  
public void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
}
```

3. onCreateView()

Creates the Fragment UI.

```
@Override  
  
public View onCreateView(  
    LayoutInflater inflater,  
    ViewGroup container,  
    Bundle savedInstanceState) {  
    return inflater.inflate(  
        R.layout.fragment_home,  
        container,  
        false);  
}
```

4. onViewCreated()

Initialize Views.

```
@Override  
  
public void onViewCreated(  
    View view,  
    Bundle savedInstanceState) {  
    super.onViewCreated(view, savedInstanceState);  
}
```

5. onStart()

Fragment becomes visible.

6. onResume()

User starts interacting.

7. onPause()

Fragment partially hidden.

8. onStop()

Fragment not visible.

9. onDestroyView()

Destroy UI.

10. onDestroy()

Fragment destroyed.

11. onDetach()

Fragment detached from Activity.

FragmentManager

Definition

FragmentManager manages Fragments.

Functions:

- Add Fragment
- Remove Fragment
- Replace Fragment
- Find Fragment

Example:

```
FragmentManager manager=
getSupportFragmentManager();
```

FragmentTransaction

Definition

Used to perform operations on Fragments.

Methods:

Method	Purpose
add()	Add Fragment
replace()	Replace Fragment
remove()	Remove Fragment
hide()	Hide Fragment
show()	Show Fragment
commit()	Save changes

Example

```
FragmentTransaction transaction=
getSupportFragmentManager()
.beginTransaction();
transaction.replace(
R.id.container,
new ProfileFragment());
transaction.commit();
```

Passing Data Between Activity and Fragment

Activity → Fragment

```
Bundle bundle=new Bundle();  
bundle.putString("name","Ali");  
HomeFragment fragment=new HomeFragment();  
fragment.setArguments(bundle);
```

Inside Fragment:

```
String name=  
getArguments().getString("name");
```

Fragment → Activity

Use an Interface.

```
public interface OnButtonClick{  
void sendData(String name);  
}
```

The Activity implements this interface and receives data from the Fragment.

Back Stack

Normally:

```
Home  
↓  
Profile  
↓  
Settings
```

Press Back:

Settings Removed

↓

Profile

Use:

```
transaction.addToBackStack(null);
```

Fragment Container

Fragments are displayed inside a container.

```
<FrameLayout  
    android:id="@+id/fragmentContainer"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"/>
```

Complete Working

MainActivity Starts

↓

FragmentManager

↓

FragmentTransaction

↓

HomeFragment Loaded

↓

User Clicks Button

↓

ProfileFragment Loaded

↓

Back Button

↓

HomeFragment Again

Advantages of Fragments

- Reusable
 - Flexible UI
 - Better code organization
 - Easy maintenance
 - Supports tablets and phones
 - Dynamic screen changes
 - Improves user experience
-

Disadvantages

- More complex than Activities.
 - Communication between Fragments requires extra code.
 - Lifecycle is more complicated.
-

Fragment vs Activity

Feature	Fragment	Activity
Independent	No	Yes
Reusable	Yes	No
Lifecycle	Yes	Yes
UI Component	Partial	Full Screen
Lightweight	Yes	No
Hosted Inside	Activity	Android OS

Static Fragment vs Dynamic Fragment

Static Fragment	Dynamic Fragment
Defined in XML	Added using Java/Kotlin
Cannot change at runtime	Can change during runtime
Simpler	More flexible
Less commonly used	Used in modern apps

Real-Life Applications

Fragments are used in:

- Gmail (Inbox + Email Content)
 - WhatsApp (Chats, Status, Calls Tabs)
 - YouTube (Home, Shorts, Subscriptions)
 - Facebook (News Feed, Friends, Notifications)
 - Instagram (Home, Search, Reels, Profile)
 - Banking Apps (Dashboard, Accounts, Transactions)
-

Frequently Asked Exam Questions

Short Questions (2–5 Marks)

1. What is a Fragment?
 2. Differentiate between Activity and Fragment.
 3. What is FragmentManager?
 4. What is FragmentTransaction?
 5. Name different Fragment lifecycle methods.
 6. What is the purpose of `onCreateView()`?
-

Long Questions (10–15 Marks)

1. Explain Fragments with architecture and diagram.
 2. Explain the Fragment Lifecycle.
 3. Explain FragmentManager and FragmentTransaction with examples.
 4. Differentiate between Static and Dynamic Fragments.
 5. Explain how to create and display a Fragment.
-

MCQs

1. A Fragment is:

- A. A database
 - B. A reusable part of an Activity ✓
 - C. An API
 - D. A Service
-

2. Which class manages Fragments?

- A. ActivityManager
 - B. FragmentManager ✓
 - C. LayoutManager
 - D. NotificationManager
-

3. Which method creates the Fragment UI?

- A. onCreate()
 - B. onCreateView() ✓
 - C. onStart()
 - D. onResume()
-

4. Which class performs operations like add(), replace(), and remove()?

- A. FragmentLifecycle
 - B. FragmentTransaction ✓
-

C. Adapter

D. Intent

5. Which container is commonly used to host Fragments?

A. TextView

B. FrameLayout ✓

C. Button

D. RecyclerView

Viva Questions

Q1: What is a Fragment?

A Fragment is a reusable part of an Activity that has its own layout, lifecycle, and behavior.

Q2: Can a Fragment exist without an Activity?

No. A Fragment must always be hosted inside an Activity.

Q3: What is the purpose of FragmentManager?

FragmentManager manages Fragments by adding, removing, replacing, and finding them.

Q4: Which lifecycle method creates the Fragment's UI?

`onCreateView()`.

Q5: What is the difference between Static and Dynamic Fragments?

Static Fragments are declared in XML and remain fixed, while Dynamic Fragments are added or replaced at runtime using `FragmentManager`.

Quick Revision (5 Minutes Before Exam)

- **Fragment:** Reusable part of an Activity.
 - **Activity:** Full screen; **Fragment:** Partial screen.
-

- **Fragment Lifecycle:** `onAttach()` → `onCreate()` → `onCreateView()` → `onViewCreated()` → `onStart()` → `onResume()` → `onPause()` → `onStop()` → `onDestroyView()` → `onDestroy()` → `onDetach()`.
- **FragmentManager:** Manages Fragments.
- **FragmentTransaction:** Performs operations like `add()`, `replace()`, `remove()`, `hide()`, `show()`, and `commit()`.
- **Static Fragment:** Declared in XML.
- **Dynamic Fragment:** Added or replaced using Java/Kotlin.
- **Container:** Usually a `FrameLayout` hosts Fragments.

Exam Tip (10–15 Marks): Start with the definition of a Fragment, draw the architecture and lifecycle diagrams, explain `FragmentManager` and `FragmentTransaction` with code examples, compare Fragment vs Activity and Static vs Dynamic Fragments, discuss advantages, disadvantages, and real-life applications, then conclude with the importance of Fragments in modern Android development. This format is ideal for scoring maximum marks in university exams.

Today 9:10 AM

isko pahle index me likho than one by in detail describe kri jao

Bilkul! Ab hum EI-333 Mobile Application Development ko one by one detailed notes ki form mein padhenge. Har topic mein Definition + Working + Diagram + Code Example + Real-world Example + Advantages/Disadvantages + Interview/Viva + Exam Notes honge. Yeh style Punjab University ke 10–15 marks questions ke liye best hai.

collapse nhi kro detailed answer do

Great. Hum **Topic #1: Mobile Computing Domain** ko ab **full detailed exam notes** ki tarah cover karte hain.

1. Mobile Computing Domain

Introduction

Mobile Computing ek technology hai jo users ko **wireless network ke zariye kisi bhi waqt aur kisi bhi jagah data access karne** ki facility deti hai.

Simple words mein:

Mobile Computing ka matlab hai mobile devices (smartphones, tablets, laptops) ki madad se internet aur data ko access karna without physical cable connection.

Aaj ke zamane mein WhatsApp, Facebook, YouTube, Google Maps, Careem, Uber aur Banking Apps sab Mobile Computing ki examples hain.

Definition

Mobile Computing is a technology that enables transmission of data, voice, and video through wireless devices without requiring a fixed physical connection.

Basic Concept

Traditional Computing:

Computer
|
|

Wired Network
|
Internet

Mobile Computing:

Mobile Device
|
|
Wireless Network
|
Internet

Yahan user move bhi kar sakta hai aur network bhi use kar sakta hai.

Components of Mobile Computing

Mobile Computing ke 3 major components hote hain.

Mobile Computing
|

| | |
Mobile Mobile Mobile
Hardware Software Communication

1. Mobile Hardware

Ye physical devices hote hain.

Examples:

- Smartphone
- Tablet
- Smart Watch
- PDA
- Laptop

Features:

- Processor
- RAM
- Battery
- Sensors

- Camera
- Storage

Example:

Samsung Galaxy S24

Contains:

- CPU
- RAM
- Display
- Battery
- Camera

Sab hardware components hain.

2. Mobile Software

Ye operating system aur applications hoti hain.

Examples:

Operating Systems

- Android
- iOS

Applications

- WhatsApp
- Facebook
- Instagram
- Banking Apps

Example:

Jab WhatsApp install karte hain to WhatsApp software hai.

3. Mobile Communication

Communication system jo devices ko connect karta hai.

Examples:

- Wi-Fi
- Bluetooth
- NFC
- 3G
- 4G
- 5G

Example:

Aap WhatsApp pe message bhejte hain.

Phone → Internet → WhatsApp Server → Receiver

Ye Mobile Communication hai.

Architecture of Mobile Computing

User

↓

Mobile Device

↓

Wireless Network

↓

Internet

↓

Application Server

↓

Database Server

Working of Mobile Computing

Step 1:

User mobile app open karta hai.

Example:

FoodPanda

↓

Step 2:

Request internet ke through server tak jati hai.

↓

Step 3:

Server database se data fetch karta hai.

↓

Step 4:

Response user ko bhej diya jata hai.

↓

Step 5:

User screen par result dekh leta hai.

Characteristics of Mobile Computing

1. Mobility

User move kar sakta hai.

Example:

Bus mein baith kar YouTube dekhna.

2. Portability

Device easily carry ki ja sakti hai.

Example:

Smartphone pocket mein aa jata hai.

3. Connectivity

Har waqt network se connected rehna.

Example:

4G ya Wi-Fi.

4. Interactivity

Real-time communication.

Example:

WhatsApp Chat.

5. Location Awareness

User ki location detect kar sakta hai.

Example:

Google Maps.

6. Personalization

User-specific content.

Example:

Instagram feed.

Types of Mobile Computing

1. Personal Mobile Computing

Single user ke liye.

Examples:

- Smartphone
 - Tablet
-

2. Enterprise Mobile Computing

Business ke liye.

Examples:

- Mobile Banking
 - Inventory Apps
-

3. Cloud Mobile Computing

Cloud services use karna.

Examples:

- Google Drive
 - Dropbox
-

Technologies Used in Mobile Computing

Wi-Fi

Wireless internet connection.

Range:

20–100 meters

Example:

Home Wi-Fi

Bluetooth

Short-distance communication.

Range:

10 meters

Example:

AirPods connection

NFC

Near Field Communication

Range:

4 cm

Example:

Google Pay

GPS

Location tracking

Example:

Google Maps

Cellular Network

- 2G
- 3G
- 4G
- 5G

Example:

Mobile Data

Applications of Mobile Computing

1. Mobile Banking

Examples:

- JazzCash
- Easypaisa
- HBL App

Functions:

- Money Transfer
 - Bill Payment
-

2. E-Commerce

Examples:

- Daraz
 - Amazon
-

Functions:

- Online Shopping
-

3. Social Media

Examples:

- Facebook
 - Instagram
 - TikTok
-

4. Healthcare

Examples:

- Online Doctor Consultation
 - Health Monitoring Apps
-

5. Education

Examples:

- Coursera
 - Udemy
 - Google Classroom
-

6. Navigation

Examples:

- Google Maps
 - Careem
 - Uber
-

7. Entertainment

Examples:

- YouTube
 - Netflix
 - Spotify
-

Advantages of Mobile Computing

1. Anywhere Access

Kahin se bhi data access kar sakte hain.

2. Real-Time Communication

Instant messaging possible hai.

3. Increased Productivity

Office work remotely kiya ja sakta hai.

4. Easy Information Sharing

Files quickly share ki ja sakti hain.

5. Better Customer Service

Business apps users ko fast services deti hain.

Disadvantages of Mobile Computing

1. Security Risks

Data theft ho sakti hai.

Example:

Phone hack ho jana.

2. Battery Limitation

Battery jaldi drain hoti hai.

3. Network Dependency

Internet na ho to services kaam nahi karti.

4. Small Screen Size

Mobile screen PC se choti hoti hai.

5. Data Cost

Mobile internet expensive ho sakta hai.

Mobile Computing vs Traditional Computing

Mobile Computing	Traditional Computing
Wireless	Mostly Wired
Portable	Fixed
Mobility Available	No Mobility
Battery Powered	Direct Power
Anywhere Access	Limited Access

Real World Example: Uber

Uber Mobile Computing ka perfect example hai.

Working:

Customer App

↓

Internet

↓

Uber Server

↓

Driver App

↓

Ride Confirmation

Features Used:

- GPS
- Internet
- Mobile App
- Cloud Database

Sab Mobile Computing ke components hain.

Viva Questions

Q1: What is Mobile Computing?

Mobile Computing is a technology that allows users to access data and services through wireless mobile devices while moving from one location to another.

Q2: What are the three components of Mobile Computing?

- Mobile Hardware

- Mobile Software
 - Mobile Communication
-

Q3: Give examples of Mobile Computing.

- WhatsApp
 - Uber
 - FoodPanda
 - Google Maps
 - Mobile Banking
-

Q4: What is Mobility?

Mobility means users can move freely while remaining connected to the network.

Q5: What is the role of GPS in Mobile Computing?

GPS helps determine the user's geographical location and provides navigation services.

Exam Question (10–15 Marks)

Explain Mobile Computing Domain with architecture, components, characteristics, advantages, disadvantages, and real-world applications.

☞ Next topic should be **Comparison of Different Mobile Platforms (Android vs iOS vs Windows Phone)** in complete detail with diagrams, tables, and real-world examples.

next

2. Comparison of Different Mobile Platforms (Android vs iOS vs Windows Phone)

This is one of the most important topics of **Mobile Application Development (EI-333)**. In exams, it is frequently asked as:

"Compare different mobile platforms."

"Differentiate between Android and iOS."

"Explain advantages and disadvantages of mobile platforms."

What is a Mobile Platform?

A **Mobile Platform** is a combination of:

- Operating System
- Hardware Support
- Application Framework
- Development Tools
- User Interface

that allows mobile applications to run on mobile devices.

Examples:

- Android
 - iOS
 - Windows Phone
-

Evolution of Mobile Platforms

1990s

Basic Phones

↓

Symbian

↓

BlackBerry OS

↓

Android (2008)

↓

iOS

↓

Windows Phone

↓

Modern Mobile Platforms

Major Mobile Platforms

There are three major mobile platforms:

Mobile Platforms

|

├─ Android

├─ iOS

└─ Windows Phone

1. Android Platform

Introduction

Android is an open-source mobile operating system developed by **Google**.

It is based on the Linux Kernel.

Launched:

2008

Features of Android

- Open Source
- Free
- Highly Customizable
- Supports Multitasking
- Large App Ecosystem
- Google Services Integration

Programming Languages

Android Apps can be developed using:

- Java
- Kotlin
- C++
- Dart (Flutter)

Development Tool

Android Studio

Official IDE for Android Development.

App Store

Google Play Store

Contains millions of applications.

Android Architecture

Applications

↓

Application Framework

↓

Android Runtime (ART)

↓

Native Libraries

↓

Linux Kernel

Examples of Android Phones

- Samsung
- Oppo
- Vivo
- Xiaomi
- OnePlus
- Realme

Advantages of Android

Open Source

Developers can modify source code.

Large User Base

Most smartphones use Android.

Customization

Themes and launchers supported.

Affordable Devices

Available in every price range.

Disadvantages of Android

Security Issues

Open nature increases malware risk.

Fragmentation

Many device versions exist.

Performance Differences

Different hardware causes inconsistent performance.

Real World Example

Samsung Galaxy S24 uses Android.

2. iOS Platform

Introduction

iOS is Apple's mobile operating system.

Developed by:

Apple Inc.

Introduced:

2007

Initially launched with iPhone.

Features of iOS

- Secure
 - Fast
 - Stable
 - High Performance
 - Smooth User Experience
-

Programming Languages

iOS development uses:

- Swift
- Objective-C

Development Tool

Xcode

Official Apple IDE.

App Store

Apple App Store

iOS Architecture

Applications

↓

Cocoa Touch

↓

Media Layer

↓

Core Services

↓

Core OS

Examples of iOS Devices

- iPhone
- iPad

Advantages of iOS

Better Security

Apple strictly controls apps.

High Performance

Optimized hardware and software.

Better User Experience

Consistent interface.

Fast Updates

All supported devices receive updates together.

Disadvantages of iOS

Expensive Devices

iPhones are costly.

Closed Source

Cannot modify system.

Less Customization

Limited user control.

Real World Example

iPhone 15 Pro uses iOS.

3. Windows Phone Platform

Introduction

Windows Phone was developed by Microsoft.

Released:

2010

Now officially discontinued.

Features

- Live Tiles Interface
 - Microsoft Services Integration
 - Easy Office Support
-

Programming Languages

- C#
 - XAML
-

Development Tool

Visual Studio

App Store

Microsoft Store

Advantages

Easy Microsoft Integration

Works well with Office.

Simple Interface

Easy navigation.

Good Performance

Required less hardware.

Disadvantages

Small App Ecosystem

Few apps available.

Limited Support

Developers stopped creating apps.

Discontinued Platform

No longer actively maintained.

Real World Example

Nokia Lumia Series

Example:

Nokia Lumia 950

Android vs iOS vs Windows Phone

Feature	Android	iOS	Windows Phone
Developer	Google	Apple	Microsoft
Launch Year	2008	2007	2010
Source Code	Open Source	Closed Source	Closed Source
Programming Language	Java, Kotlin	Swift, Objective-C	C#, XAML
IDE	Android Studio	Xcode	Visual Studio
App Store	Play Store	App Store	Microsoft Store
Security	Medium	High	Medium
Customization	High	Low	Medium
Cost	Low to High	High	Medium
Apps Available	Millions	Millions	Few
Popularity	Very High	High	Low

Market Share Comparison

Android ≈ 70%

iOS ≈ 29%

Others ≈ 1%

Android dominates the global smartphone market.

Security Comparison

Android

Security Level:

★ ★ ★

Risk:

Higher malware risk due to open ecosystem.

iOS

Security Level:

★ ★ ★ ★ ★

Risk:

Very low because Apple reviews apps.

Windows Phone

Security Level:

★ ★ ★

Risk:

Moderate.

Development Cost Comparison

Android

Low Cost

↓

iOS

Medium Cost

↓

Windows Phone

Low Cost

Performance Comparison

iOS

Best Optimization

↓

Android

Depends on Hardware

↓

Windows Phone

Good Optimization

Application Development Comparison

Android

Build APK

↓

Upload to Play Store

↓

Publish App

iOS

Build IPA

↓

App Store Review

↓

Publish App

Windows Phone

Build Package



Microsoft Store



Publish App

Which Platform is Best?

Android

Best for:

- Wide Audience
 - Low Budget Devices
 - Open Source Development
-

iOS

Best for:

- Premium Applications
 - Better Security
 - High Revenue Apps
-

Windows Phone

Best for:

- Educational Understanding
- Historical Study

(Not used in modern development)

Real World Example

Suppose you develop a Food Delivery App.

Android

Can reach millions of users.

Examples:

- FoodPanda
 - Daraz
-

iOS

Used by premium users.

Examples:

- Uber
 - Airbnb
-

Windows Phone

Not recommended because platform is discontinued.

Advantages of Mobile Platforms

- Mobility
 - Connectivity
 - Portability
 - Better Communication
 - Large App Ecosystem
-

Challenges of Mobile Platforms

- Security Threats
 - Battery Consumption
 - Device Fragmentation
 - Privacy Issues
 - Frequent Updates
-

Viva Questions

Q1: What is a Mobile Platform?

A Mobile Platform is a software and hardware environment that supports mobile application execution.

Q2: Which mobile platform is open source?

Android.

Q3: Which company developed iOS?

Apple Inc.

Q4: Which language is mainly used for Android development?

Java and Kotlin.

Q5: Which language is mainly used for iOS development?

Swift and Objective-C.

Q6: Which mobile platform is discontinued?

Windows Phone.

MCQs

1. Android is developed by:

- A. Microsoft
- B. Apple
- C. Google ✓
- D. Nokia

2. iOS applications are mainly written in:

- A. Java
- B. Swift ✓
- C. Python
- D. PHP

3. Which platform is open source?

- A. Android ✓
- B. iOS
- C. Windows Phone
- D. Symbian

4. Official IDE for Android development?

- A. Xcode

- B. Eclipse
 - C. Android Studio ✓
 - D. Visual Studio
-

5. Which platform has the highest security?

- A. Android
 - B. Windows Phone
 - C. iOS ✓
 - D. Symbian
-

Exam Question (10–15 Marks)

Compare Android, iOS, and Windows Phone platforms. Discuss their architecture, programming languages, development tools, advantages, disadvantages, and real-world applications.

Next Topic:

Revision of Java for Android Development

- OOP Concepts (Class, Object, Inheritance, Polymorphism)
- Variables & Data Types
- Constructors
- Arrays
- Exception Handling
- Collections
- Threads
- Java Code Examples used in Android Apps

(Ye topic Android development ki foundation hai aur exams mein bohat important hai.)

3. Revision of Java for Android Development (Complete Detailed Notes)

Before learning Android, it is very important to understand Java because Android applications were traditionally developed using Java. Most Android concepts such as Activities, Intents, Services, Adapters, RecyclerView, SQLite, etc. are based on Java concepts.

What is Java?

Java is a high-level, object-oriented programming language developed by Sun Microsystems (now Oracle) in 1995.

Java follows:

Write Once, Run Anywhere (WORA)

Meaning:

A Java program can run on any platform that has a Java Virtual Machine (JVM).

Why Java for Android?

Android originally used Java as its primary language.

Reasons:

- Easy to learn
 - Object-Oriented
 - Platform Independent
 - Secure
 - Large Community Support
 - Rich Libraries
-

Structure of a Java Program

Example:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Explanation

```
public class Main
```

Class declaration.

```
public static void main()
```

Program starting point.

```
System.out.println()
```

Used to print output.

Output:

```
Hello World
```

Variables in Java

Definition

Variables are memory locations used to store data.

Example:

```
int age = 20;  
  
String name = "Ali";
```

Types of Variables

Local Variable

Inside method.

```
void show(){  
    int x = 10;  
}
```

Instance Variable

Inside class but outside method.

```
class Student{  
    int age;  
}
```

Static Variable

Shared by all objects.

```
static int count = 0;
```

Data Types in Java

Primitive Data Types

Type	Size	Example
byte	1 byte	10
short	2 bytes	100
int	4 bytes	500
long	8 bytes	1000
float	4 bytes	12.5f
double	8 bytes	20.56

Type	Size	Example
char	2 bytes	'A'
boolean	1 bit	true

Example

```
int age = 20;

double salary = 50000.50;

char grade = 'A';

boolean pass = true;
```

Operators in Java

Arithmetic Operators

```

+
-
*
/
%
```

Example:

```
int a = 10;
int b = 5;

System.out.println(a+b);
```

Output:

```
15
```

Relational Operators

```

==
!=
>
<
>=
<=
```

Example:

```
System.out.println(10>5);
```

Output:

```
true
```

Logical Operators

```
&&  
||  
!
```

Example:

```
if (age>18 && age<60)
```

Conditional Statements

If Statement

```
int age = 20;  
  
if (age>=18) {  
  
    System.out.println("Adult");  
  
}
```

Output:

```
Adult
```

If-Else

```
if (age>=18)  
  
    System.out.println("Adult");  
  
else  
  
    System.out.println("Child");
```

Switch Statement

```
int day = 2;

switch(day) {

case 1:

System.out.println("Monday");

break;

case 2:

System.out.println("Tuesday");

break;

}
```

Output:

Tuesday

Loops

Loops repeat statements.

For Loop

```
for(int i=1;i<=5;i++){

System.out.println(i);

}
```

Output:

1
2
3
4
5

While Loop

```
int i=1;

while(i<=5){

System.out.println(i);

i++;

}
```

Do While Loop

```
int i=1;

do{

System.out.println(i);

i++;

}while(i<=5);
```

Arrays

Definition

Array stores multiple values of the same type.

Example

```
int numbers[] = {10,20,30,40};
```

Access Elements

```
System.out.println(numbers[0]);
```

Output:

```
10
```

Loop Through Array

```
for(int i=0;i<numbers.length;i++){  
    System.out.println(numbers[i]);  
}
```

Object-Oriented Programming (OOP)

Java is an Object-Oriented Language.

Main OOP Concepts:

OOP

|

├─ Class

├─ Object

├─ Inheritance

├─ Polymorphism

├─ Encapsulation

└─ Abstraction

Class

Definition

A class is a blueprint for creating objects.

Example:

```
class Student{  
  
    String name;  
  
    int age;  
  
}
```

Object

Object is an instance of a class.

```
Student s1 = new Student();
```

Real World Example

Class:

Car

Object:

Honda Civic

Toyota Corolla

Constructor

Definition

A constructor initializes objects.

Example

```
class Student{  
    Student() {  
        System.out.println("Constructor Called");  
    }  
}
```

Object Creation

```
Student s = new Student();
```

Output:

Types of Constructor

Default Constructor

```
Student() {  
  
}
```

Parameterized Constructor

```
Student(String name) {  
  
    this.name = name;  
  
}
```

Inheritance

Definition

Inheritance allows one class to acquire properties of another class.

Example

```
class Animal {  
  
    void eat() {  
  
        System.out.println("Eating");  
  
    }  
  
}  
  
class Dog extends Animal {  
  
}
```

Usage

```
Dog d = new Dog();  
d.eat();
```

Output:

Eating

Real World Example

Animal

↓

Dog

↓

German Shepherd

Polymorphism

Definition

One method can have multiple forms.

Method Overloading

```
class Maths{  
  
    int add(int a,int b){  
  
        return a+b;  
  
    }  
  
    double add(double a,double b){  
  
        return a+b;  
  
    }  
  
}
```

Method Overriding

```
class Animal{  
  
void sound(){  
  
System.out.println("Animal Sound");  
  
}  
  
}  
  
class Dog extends Animal{  
  
void sound(){  
  
System.out.println("Bark");  
  
}  
  
}
```

Encapsulation

Definition

Wrapping data and methods together.

Example

```
class Student{  
  
private int age;  
  
public void setAge(int age){  
  
this.age = age;  
  
}  
  
public int getAge(){  
  
return age;  
  
}  
  
}
```

Abstraction

Definition

Showing essential details while hiding implementation.

Example

```
abstract class Vehicle{  
    abstract void start();  
}
```

Exception Handling

Definition

Exception is a runtime error.

Without Exception Handling

```
int a = 10/0;
```

Program crashes.

Using Try Catch

```
try{  
    int a = 10/0;  
}  
catch(Exception e){  
    System.out.println("Error");  
}
```

Output:

Error

Collections Framework

Used for dynamic data storage.

ArrayList

```
ArrayList<String> list=  
new ArrayList<>();  
list.add("Ali");  
list.add("Ahmed");
```

Print Data

```
System.out.println(list);
```

Output:

```
[Ali, Ahmed]
```

HashMap

Stores key-value pairs.

```
HashMap<Integer,String> map=  
new HashMap<>();  
map.put(1,"Ali");  
map.put(2,"Ahmed");
```

Output

```
1=Ali
```

```
2=Ahmed
```

Threads

Definition

A thread is a lightweight process.

Used for background tasks.

Example

```
class MyThread extends Thread{  
    public void run(){  
        System.out.println("Running");  
    }  
}
```

Start Thread

```
MyThread t = new MyThread();  
t.start();
```

Output

Running

Java in Android

Example Button Click

```
Button btn;  
  
btn = findViewById(R.id.btn);
```

```
btn.setOnClickListener(v->{  
  
    Toast.makeText(  
  
        this,  
  
        "Button Clicked",  
  
        Toast.LENGTH_SHORT  
  
    ).show();  
  
});
```

Real World Example (Student App)

```
class Student{  
  
    String name;  
  
    int marks;  
  
    Student(String name,int marks){  
  
        this.name = name;  
  
        this.marks = marks;  
  
    }  
  
}
```

Create Object

```
Student s1 =  
  
new Student("Ali",90);
```

Output

```
Student Name = Ali  
  
Marks = 90
```

Java vs Kotlin

Java	Kotlin
Older Language	Modern Language
More Code	Less Code
Verbose	Concise
Widely Used	Official Android Language
Easy Learning Resources	Faster Development

Advantages of Java

- Platform Independent
 - Object-Oriented
 - Secure
 - Robust
 - Multithreading Support
 - Large Community
-

Disadvantages

- More Boilerplate Code
 - Higher Memory Usage
 - Slower than C++
-

Viva Questions

Q1: What is Java?

Java is a high-level object-oriented programming language used for Android application development.

Q2: What is JVM?

JVM (Java Virtual Machine) executes Java bytecode.

Q3: What is OOP?

Object-Oriented Programming is a programming paradigm based on objects and classes.

Q4: What is Inheritance?

Inheritance allows one class to acquire properties and methods of another class.

Q5: What is Polymorphism?

Polymorphism allows methods to behave differently based on context.

Q6: What is Exception Handling?

Exception handling manages runtime errors using try-catch blocks.

Important Exam Questions

Short Questions (5 Marks)

1. What is Java?
 2. Explain Constructor.
 3. Define Inheritance.
 4. What is Polymorphism?
 5. What is an Array?
 6. Explain Exception Handling.
-

Long Questions (10–15 Marks)

1. Explain OOP concepts with examples.
 2. Explain Inheritance and Polymorphism with Java programs.
 3. Discuss Java Collections Framework.
 4. Explain Exception Handling in Java.
 5. Explain Java features and its role in Android Development.
-

Next Topic (Very Important)

Android Platform

- History of Android
- Features of Android
- Android Versions
- Android Architecture (Linux Kernel, Libraries, ART, Framework, Apps)
- Dalvik Virtual Machine (DVM)
- Android Runtime (ART)
- Complete Architecture Diagram
- Real-world Examples
- Exam Questions & MCQs

This is one of the most important 10–15 marks topics in EI-333.

4. Android Platform (Complete Detailed Notes)

This is one of the most important topics of Mobile Application Development (EI-333) and is frequently asked in 10–15 marks questions.

Introduction to Android

Android is an open-source mobile operating system developed by Google. It is based on the Linux Kernel and is designed primarily for smartphones, tablets, TVs, smartwatches, and other smart devices.

Simple Definition:

Android is a Linux-based operating system used to develop and run mobile applications on smart devices.

Real-World Examples:

- Samsung Galaxy
- OnePlus
- Xiaomi
- Oppo
- Vivo
- Realme
- Google Pixel

History of Android

Year	Event
2003	Android Inc. founded by Andy Rubin.
2005	Google acquired Android Inc.
2007	Android announced publicly.
2008	First Android phone launched (HTC Dream / T-Mobile G1).
2013+	Android became the world's most popular mobile OS.
Today	Used on billions of devices worldwide.

Why Android Became Popular?

- Open Source
- Free for manufacturers
- Supports millions of apps
- Easy customization
- Large developer community
- Google services integration
- Available on low-end and high-end devices

Features of Android

1. Open Source

Manufacturers can modify Android according to their needs.

Example:

- Samsung One UI
- Xiaomi HyperOS
- OxygenOS

2. Multitasking

Multiple applications can run simultaneously.

Example:

- Listening to Spotify while using WhatsApp.

3. Connectivity

Supports various communication technologies.

- Wi-Fi
- Bluetooth
- NFC
- USB
- 4G/5G

4. Rich Multimedia Support

Supports:

- Audio
- Video
- Camera
- Animations
- Graphics

5. Security

Provides:

- App sandboxing
- Permissions
- Encryption
- Google Play Protect

6. Notifications

Users receive updates even when the app is closed.

Examples:

- WhatsApp message
- Gmail email
- FoodPanda order

7. Location Services

Uses GPS for navigation.

Example:

- Google Maps
- Uber
- Careem

Android Versions

Version	Codename
1.5	Cupcake
1.6	Donut
2.1	Eclair
2.2	Froyo
2.3	Gingerbread
3.x	Honeycomb
4.x	Ice Cream Sandwich
4.1–4.3	Jelly Bean
4.4	KitKat
5.x	Lollipop
6.0	Marshmallow
7.x	Nougat
8.x	Oreo
9	Pie
10	Android 10
11	Android 11
12	Android 12
13	Android 13
14	Android 14
15	Android 15

Android Software Stack

Android consists of multiple layers.

Applications → Application Framework → Android Runtime (ART) → Native Libraries → Linux Kernel

This is called the Android Architecture.

Android Architecture (Very Important)

Applications

↓

Application Framework

↓

Android Runtime (ART)

Native Libraries



Linux Kernel

1. Applications Layer

Contains all user applications.

Examples:

- WhatsApp
- Instagram
- YouTube
- Gmail
- Google Maps
- Your Android App

Responsibilities:

- Provide user interface
- Handle user interactions
- Use framework APIs

2. Application Framework Layer

Provides high-level services for app developers.

Major Services:

Service	Purpose
Activity Manager	Manages Activities
Window Manager	Manages windows
Package Manager	Manages app installation
Notification Manager	Shows notifications
Location Manager	GPS services
Resource Manager	Access resources
Telephony Manager	Phone services
Content Providers	Data sharing

Real-World Example:

When WhatsApp shows a notification, it uses NotificationManager.

3. Android Runtime (ART)

ART executes Android applications.

Features:

- Runs app bytecode
- Garbage Collection
- Better performance
- Lower battery consumption

ART Working:

Java/Kotlin Code

↓

Bytecode

↓

ART Compiles

↓

Machine Code

↓

CPU Executes

4. Native Libraries

Written mostly in C/C++.

Examples:

- SQLite
- OpenGL ES
- WebKit
- SSL
- Media Framework

Real-World Example:

When YouTube plays a video, Android uses Media Framework libraries.

5. Linux Kernel

This is the foundation of Android.

Responsibilities:

- Process Management
- Memory Management
- Security
- Device Drivers
- Power Management
- Networking

Example:

Camera driver and Wi-Fi driver are handled by the Linux Kernel.

Dalvik Virtual Machine (DVM)

Older Android versions used Dalvik Virtual Machine.

Features:

- Optimized for mobile devices
- Low memory usage
- Runs .dex files

Working:

Java Source

↓

.class

↓

dx Tool

↓

.dex

↓

Dalvik VM

DVM vs JVM

DVM	JVM
For Android	General Java
Uses .dex	Uses .class
Low memory	Higher memory
Mobile optimized	Desktop/server oriented

ART vs Dalvik

ART	Dalvik
Faster	Slower
Better battery life	More battery usage
AOT + JIT	Mainly JIT
Used in modern Android	Used in old Android

Android Application Development Process

- Write code in Java/Kotlin.
- Build project in Android Studio.
- Generate APK.
- Install on device/emulator.
- Run application.

Code → Compile → APK → Install → Run

Simple Android Example

MainActivity.java

Output:

Hello Android

Real-World Example: WhatsApp

Layers Used:

Layer	Usage
Applications	WhatsApp UI

Layer	Usage
Framework	Notifications, Camera
ART	Runs app code
Libraries	SQLite, Media
Linux Kernel	Network, Memory

Advantages of Android

- Open Source
- Huge app ecosystem
- Affordable devices
- Customizable
- Multitasking
- Google services integration
- Large developer community

Disadvantages of Android

- Device fragmentation
- Security risks
- Different performance on different devices
- Manufacturer bloatware
- Delayed updates on some phones

Viva Questions

- What is Android? → A Linux-based mobile operating system developed by Google.
- Which kernel does Android use? → Linux Kernel.
- What is ART? → Android Runtime that executes Android applications.
- What is Dalvik VM? → An older virtual machine used by Android to run .dex files.
- Name any four framework services. → Activity Manager, Package Manager, Notification Manager, Location Manager.
- What is an APK? → Android application package used to install apps.

MCQs

1. Android is based on:

A. Windows

B. Linux Kernel ✓

C. macOS

D. UNIX

2. Google acquired Android Inc. in:

A. 2003

B. 2005 ✓

C. 2007

D. 2008

3. Which runtime is used in modern Android?

A. JVM

B. Dalvik

C. ART ✓

D. CLR

4. Which layer manages notifications?

A. Kernel

B. Application Framework ✓

C. Libraries

D. Hardware

5. SQLite belongs to:

A. Application Layer

B. Native Libraries ✓

C. Kernel

D. Runtime

Important Exam Question (15 Marks)

Explain Android Platform in detail. Discuss its history, features, versions, Dalvik VM, Android Runtime (ART), and Android Architecture with a neat diagram.

Next Topic:

Android Components (Very Important)

- Activity
- Service
- Broadcast Receiver
- Content Provider
- Architecture
- Lifecycle
- Code Examples
- Real-World Examples
- Exam Questions & Viva

5. Android Components (Complete Detailed Notes)

Android applications are built using four core components. These components are the foundation of every Android app and are almost always asked in Punjab University exams (10–15 marks).

What are Android Components?

Definition:

Android Components are the building blocks of an Android application that define how the app behaves and interacts with users and the Android system.

The four main components are:

- Activity
- Service
- Broadcast Receiver
- Content Provider

Android Components Architecture

Android Application

|

|— Activity

|— Service

└─ Broadcast Receiver

└─ Content Provider

Each component performs a different task.

1. Activity

Definition

An Activity represents a single screen with a user interface.

Simple words:

Activity is the screen that the user sees and interacts with.

Examples:

- Login Screen
- Registration Screen
- Home Screen
- Settings Screen

Activity Lifecycle

onCreate()

↓

onStart()

↓

onResume()

↓

(Running)

↓

onPause()

↓

onStop()

↓

onDestroy()

Create an Activity

MainActivity.java

Explanation:

- `AppCompatActivity` → Base activity class.
- `onCreate()` → Called when activity is created.
- `setContentView()` → Loads the XML layout.

XML Layout

activity_main.xml

Output:

Hello Android

Start Another Activity

Flow:

MainActivity

↓

Intent

↓

SecondActivity

Real-World Example

WhatsApp:

- Chat List = Activity
- Chat Screen = Activity
- Status Screen = Activity
- Settings Screen = Activity

Advantages

- Provides UI.
- Handles user interaction.
- Easy navigation between screens.
- Supports lifecycle management.

2. Service

Definition

A Service performs long-running operations in the background without a user interface.

Simple words:

A Service works behind the scenes while the user may be using another app.

Examples:

- Music Playback
- File Download
- Location Tracking
- Backup Service

Types of Services

1. Started Service

Runs until stopped.

2. Bound Service

Allows components to communicate with the service.

3. Foreground Service

Shows a persistent notification.

Example:

- Google Maps Navigation
- Spotify Music

Create a Service

Start Service

Stop Service

Service Lifecycle

onCreate()

↓

onStartCommand()

↓

(Running)

↓

onDestroy()

Real-World Example

Spotify:

- User locks phone.
- Music continues playing.
- Background playback is handled by a Service.

3. Broadcast Receiver

Definition

A Broadcast Receiver responds to system-wide or application-wide broadcast messages.

Simple words:

It listens for important events and reacts when they occur.

Examples:

- Battery Low
- Phone Boot Completed
- Network Connected
- SMS Received

Create Broadcast Receiver

Register in Manifest

Working

System Event



Broadcast Sent



Broadcast Receiver



Action Performed

Real-World Example

Battery Saver App:

- System sends `BATTERY_LOW`.
- Receiver receives it.
- App turns on power-saving mode.

4. Content Provider

Definition

A Content Provider manages and shares application data securely with other applications.

Simple words:

It is a standard way to share data between apps.

Examples:

- Contacts
- Call Logs
- Media Files
- Calendar

Content Provider Architecture

App A



Content Provider



Database



App B

CRUD Operations

Operation	Method
-----------	--------

Create	<code>insert()</code>
--------	-----------------------

Read	<code>query()</code>
------	----------------------

Update	<code>update()</code>
--------	-----------------------

Delete	<code>delete()</code>
--------	-----------------------

Simple Example

This code reads phone contacts.

Real-World Example

Truecaller:

- Requests permission.
- Reads contacts through Content Provider.
- Displays caller information.

How Components Communicate?

Android components usually communicate using Intents.

Activity → Activity

Activity → Service

Broadcast → Receiver

App → Content Provider

Complete Workflow

User Opens App

↓

Activity Starts

↓

Activity Starts Service

↓

Service Downloads Data

↓

Broadcast Sent

↓

Receiver Receives Event

↓

Data Stored in Content Provider

↓

Other Apps Can Access Data

Activity vs Service

Activity	Service
Has UI	No UI
User interacts directly	Works in background
Represents a screen	Performs long tasks

Broadcast Receiver vs Content Provider

Broadcast Receiver	Content Provider
Receives events	Shares data
No data storage	Provides CRUD operations
Event-driven	Data-driven

Advantages of Android Components

- Modular application design.
- Code reusability.
- Background processing.
- Inter-app communication.
- Secure data sharing.
- Better user experience.

Frequently Asked Exam Questions

Short Questions (2–5 Marks)

- What is an Activity?
- What is a Service?
- What is a Broadcast Receiver?
- What is a Content Provider?
- Write any two methods of Content Provider.
- Differentiate between Activity and Service.

Long Questions (10–15 Marks)

- Explain Android Components with a neat diagram.
- Explain Activity lifecycle with suitable examples.
- Differentiate between all four Android components.
- Explain Content Provider and its CRUD operations.
- Explain Service and its lifecycle with examples.

MCQs

- Which component provides a user interface? → Activity ✓
- Which component performs background tasks? → Service ✓
- Which component responds to system events? → Broadcast Receiver ✓
- Which component shares data between apps? → Content Provider ✓
- Which class is used to navigate between Activities? → Intent ✓

Quick Revision (5 Minutes Before Exam)

- Activity: UI screen.

- Service: Background task.
- Broadcast Receiver: Receives broadcasts/events.
- Content Provider: Shares and manages data.
- Communication: Usually through Intents.
- Remember: Activity = Screen, Service = Background, Receiver = Event, Provider = Data.

Next Topic:

Activity Lifecycle in Complete Detail (Very Important)

- `onCreate()`
- `onStart()`
- `onResume()`
- `onPause()`
- `onStop()`
- `onRestart()`
- `onDestroy()`
- Lifecycle Diagram
- Code Example
- Real-World Example
- Interview/Viva Questions
- Exam Notes

6. Activity Lifecycle in Android (Complete Detailed Notes)

Activity Lifecycle is one of the **most important topics** in Android Development. It is almost always asked in exams because every Android application screen follows this lifecycle.

Without understanding the lifecycle, developers cannot properly manage:

- User interactions
- Memory
- Resources
- App state
- Performance

What is an Activity?

An **Activity** represents a single screen of an Android application.

Examples:

- Login Screen
- Registration Screen
- Home Screen
- Settings Screen
- Profile Screen

Every screen you see in an Android app is usually an Activity.

What is Activity Lifecycle?

Definition

The Activity Lifecycle is a sequence of callback methods that Android calls as an Activity is created, displayed, paused, stopped, restarted, and destroyed.

In simple words:

Activity Lifecycle tells us what happens when an Activity starts, runs, pauses, resumes, or closes.

Why Activity Lifecycle is Important?

Android devices have:

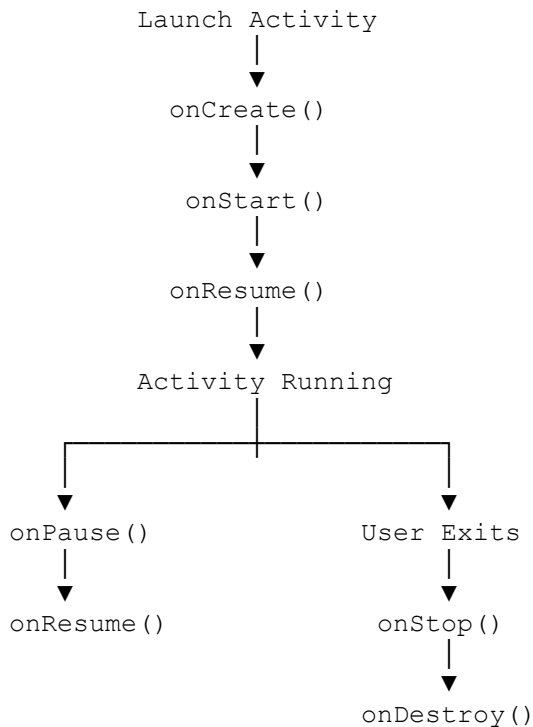
- Limited RAM
- Limited battery
- Multiple running applications

Android manages Activities automatically.

Lifecycle helps developers:

- Save user data
 - Release resources
 - Prevent crashes
 - Improve performance
-

Complete Activity Lifecycle Diagram



Lifecycle Methods

Android provides 7 important lifecycle methods:

`onCreate()`

`onStart()`

`onResume()`

`onPause()`

`onStop()`

`onRestart()`

`onDestroy()`

1. onCreate()

Definition

Called when Activity is created for the first time.

This is the starting point of an Activity.

Purpose

Used for:

- Initializing variables
 - Connecting UI components
 - Loading layouts
 - Setting listeners
-

Example

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    Log.d("Lifecycle", "onCreate");
}
```

Real World Example

When user opens WhatsApp:

Tap WhatsApp

↓

Activity Created

↓

onCreate()

WhatsApp loads:

- Chat list
 - Toolbar
 - User information
-

2. onStart()

Definition

Called when Activity becomes visible to the user.

Activity is now appearing on screen.

Purpose

Used for:

- Register listeners
 - Prepare UI
 - Start lightweight tasks
-

Example

```
@Override
protected void onStart() {
    super.onStart();

    Log.d("Lifecycle", "onStart");
}
```

Real World Example

User opens Instagram.

Instagram screen becomes visible.

Android calls:

`onStart()`

3. onResume()

Definition

Called when Activity enters foreground and user can interact with it.

Activity is now fully active.

Purpose

Used for:

- Start camera
 - Start sensors
 - Resume videos
 - Enable user interaction
-

Example

```
@Override
protected void onResume() {
    super.onResume();

    Log.d("Lifecycle", "onResume");
}
```

Real World Example

User opens YouTube video.

Video starts playing.

`onResume()`

is executed.

Activity Running State

At this stage:

Activity Running

User can:

- Click buttons
 - Enter data
 - Scroll screen
 - Watch videos
-

4. onPause()

Definition

Called when Activity partially loses focus.

Another Activity is coming in front.

Purpose

Used for:

- Save temporary data
 - Pause animations
 - Pause video
 - Release camera
-

Example

```
@Override
protected void onPause() {
    super.onPause();

    Log.d("Lifecycle", "onPause");
}
```

Real World Example

Suppose:

You are watching YouTube

↓

Incoming Call Appears

↓

YouTube Partially Hidden

Android calls:

```
onPause()
```

Video pauses.

5. onStop()

Definition

Called when Activity is no longer visible.

Purpose

Used for:

- Release resources
 - Stop background tasks
 - Save user data
-

Example

```
@Override
```

```
protected void onStop() {
    super.onStop();

    Log.d("Lifecycle", "onStop");
}
```

Real World Example

User presses Home button.

Activity Hidden

↓

onStop()

Activity moves to background.

6. onRestart()

Definition

Called when a stopped Activity is restarted.

Purpose

Prepare Activity to return to foreground.

Example

```
@Override
protected void onRestart() {
    super.onRestart();

    Log.d("Lifecycle", "onRestart");
}
```

Real World Example

User:

Open Facebook

↓

Press Home

↓

Return to Facebook

Android calls:

`onRestart()`

7. onDestroy()

Definition

Called before Activity is destroyed permanently.

Purpose

Used for:

- Cleanup
 - Release memory
 - Close database
 - Stop threads
-

Example

```
@Override
protected void onDestroy() {
    super.onDestroy();

    Log.d("Lifecycle", "onDestroy");
}
```

Real World Example

User closes app completely.

Activity Ends

↓

onDestroy()

Complete Lifecycle Example

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Log.d("Lifecycle", "onCreate");
    }

    @Override
    protected void onStart() {
        super.onStart();
        Log.d("Lifecycle", "onStart");
    }

    @Override
    protected void onResume() {
        super.onResume();
        Log.d("Lifecycle", "onResume");
    }

    @Override
    protected void onPause() {
        super.onPause();
        Log.d("Lifecycle", "onPause");
    }

    @Override
    protected void onStop() {
        super.onStop();
        Log.d("Lifecycle", "onStop");
    }

    @Override
    protected void onRestart() {
        super.onRestart();
        Log.d("Lifecycle", "onRestart");
    }

    @Override
    protected void onDestroy() {
```

```
        super.onDestroy();  
        Log.d("Lifecycle", "onDestroy");  
    }  
}
```

Lifecycle Sequence Examples

Case 1: App Launch

```
onCreate()  
  
↓  
  
onStart()  
  
↓  
  
onResume()
```

Case 2: User Presses Home Button

```
onPause()  
  
↓  
  
onStop()
```

Case 3: User Returns to App

```
onRestart()  
  
↓  
  
onStart()  
  
↓  
  
onResume()
```

Case 4: User Closes App

```
onPause()  
  
↓
```

```
onStop()
```

↓

```
onDestroy()
```

State Saving

Suppose user enters:

```
Name = Ali
```

Phone rotates.

Activity recreates.

Without saving state:

```
Ali
```

↓

```
Lost
```

Save State Example

```
@Override
protected void onSaveInstanceState(Bundle outState) {

    outState.putString("name", "Ali");

    super.onSaveInstanceState(outState);
}
```

Restore State

```
@Override
protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    if(savedInstanceState!=null){

        String name=
```

```
        savedInstanceState.getString("name");  
    }  
}
```

Real Life Example (WhatsApp)

When user opens WhatsApp:

```
onCreate()  
  
↓  
onStart()  
  
↓  
onResume()
```

Incoming Call:

```
onPause()
```

User Returns:

```
onResume()
```

User Closes WhatsApp:

```
onPause()  
  
↓  
onStop()  
  
↓  
onDestroy()
```

Advantages of Activity Lifecycle

- Better memory management
- Better battery usage
- Efficient resource handling

- Prevent crashes
 - Preserve user data
 - Smooth user experience
-

Common Interview Questions

Q1: Which method is called first?

`onCreate()`

Q2: Which method makes Activity visible?

`onStart()`

Q3: Which method allows user interaction?

`onResume()`

Q4: Which method is called when Activity loses focus?

`onPause()`

Q5: Which method is called before Activity destruction?

`onDestroy()`

MCQs

1. First lifecycle method is:

- A. `onResume()`
- B. `onStart()`
- C. `onCreate()` ✓
- D. `onPause()`

2. Which method is called when Activity becomes visible?

- A. onCreate()
- B. onStart() ✓
- C. onDestroy()
- D. onPause()

3. Which method is called when Activity is completely hidden?

- A. onResume()
- B. onPause()
- C. onStop() ✓
- D. onRestart()

4. Which method is called before Activity is destroyed?

- A. onStop()
- B. onDestroy() ✓
- C. onStart()
- D. onResume()

Frequently Asked Exam Question

Explain Android Activity Lifecycle with a neat diagram and suitable examples. Discuss all lifecycle methods in detail.

Quick Revision (2 Minutes Before Exam)

`onCreate()` → Create Activity
`onStart()` → Visible
`onResume()` → Running
`onPause()` → Partially Hidden
`onStop()` → Fully Hidden
`onRestart()` → Restart Activity
`onDestroy()` → Destroy Activity

Next Topic (Very Important)

Intent in Android

We will cover:

- What is Intent?
- Types of Intent
- Explicit Intent
- Implicit Intent
- Passing Data Between Activities
- Bundle
- Intent Filters
- Real-world Examples
- Full Java Code
- Interview Questions
- Exam Notes

This is one of the most important Android programming topics and often appears in practical as well as theory exams.

. Intent in Android (Complete Detailed Notes)

Intent is one of the most important concepts in Android. Almost every Android application uses Intents for navigation, communication, sharing data, opening apps, making calls, sending emails, etc.

This topic is frequently asked in 5, 10, and 15 marks exams.

What is an Intent?

Definition

An Intent is a messaging object used to request an action from another Android component.

Simple Words:

Intent is used to communicate between Activities, Services, and Broadcast Receivers.

Example:

- Open another screen.
- Send data to another Activity.
- Open the camera.
- Make a phone call.
- Open a web browser.
- Share text with WhatsApp.

Intent Architecture

Activity A



Intent



Android System



Target Component



Activity B / Service / Receiver

Types of Intents

There are two main types:

- Explicit Intent
- Implicit Intent

1. Explicit Intent

Definition

An Explicit Intent specifies the exact component to open.

Simple Words:

You know exactly which Activity should be started.

Example:

LoginActivity → HomeActivity

Create Explicit Intent

Explanation:

- `this` → Current Activity.
- `SecondActivity.class` → Target Activity.
- `startActivity()` → Starts the Activity.

Complete Example

MainActivity.java

SecondActivity.java

Flow

MainActivity

↓

Button Click

↓

Intent

↓

SecondActivity

Real-World Example

WhatsApp:

- Chat List → Chat Screen
- Settings → Account Settings
- Profile → Edit Profile

These are typically opened using Explicit Intents.

2. Implicit Intent

Definition

An Implicit Intent does not specify a particular component. Instead, it describes the action to perform.

Simple Words:

Android decides which application can handle the request.

Examples:

- Open Browser
- Dial Phone Number
- Send Email
- Open Camera
- Share Text
- Open Maps

Open Website Example

Output:

Browser opens Google.

Dial Phone Number

Output:

Phone dialer opens.

Send Email

Share Text

Output:

WhatsApp, Gmail, Facebook, and other apps appear.

Real-World Example

Instagram:

- User clicks a web link.
- Instagram sends an ACTION_VIEW intent.
- Android opens the browser.

Intent Filters

Definition

An Intent Filter tells Android which Intents a component can handle.

Example in Manifest

This Activity can handle HTTPS links.

Intent Filter Working

Intent Sent

↓

Android Checks Filters

↓

Matching Component Found

↓

Component Started

Passing Data Between Activities

Using putExtra()

Sender Activity

Receiver Activity

Output:

Name = Ali

Age = 20

Real-World Example

E-Commerce App:

- Product List Activity sends product ID.
- Product Detail Activity receives the ID.
- Product information is displayed.

Bundle in Android

Definition

A Bundle stores key-value pairs.

Send Bundle

Receive Bundle

Getting Result Back from Activity

Suppose:

- Activity A opens Activity B.
- Activity B returns selected data.

Modern Approach:

- Activity Result API

Concept:

Activity A

↓

Open Activity B

↓

User Selects Data

↓

Return Result

↓

Activity A Receives Result

Common Intent Actions

Action	Purpose
ACTION_VIEW	View content
ACTION_DIAL	Open dialer
ACTION_CALL	Make a call
ACTION_SEND	Share data
ACTION_SENDTO	Send email/SMS
ACTION_PICK	Pick contact/image
ACTION_MAIN	Main entry point

Intent Flags

Flags modify Activity launch behavior.

Example:

Common Flags:

Flag	Purpose
FLAG_ACTIVITY_NEW_TASK	Start in new task
FLAG_ACTIVITY_CLEAR_TOP	Clear activities above target
FLAG_ACTIVITY_SINGLE_TOP	Reuse top activity

Explicit vs Implicit Intent

Explicit Intent	Implicit Intent
Target component specified	No target specified
Used within app	Can open other apps
Faster	Android chooses component
Example: Open HomeActivity	Example: Open Browser

Complete Real-World Scenario

FoodPanda:

- User opens app → MainActivity.
- User selects restaurant → Explicit Intent opens RestaurantActivity.
- User taps location → Implicit Intent opens Google Maps.
- User taps share → ACTION_SEND shares restaurant link.
- User places order → Order ID passed using Intent Extras.

Advantages of Intents

- Easy navigation.
- Communication between components.
- Data sharing.
- Launch system apps.
- Loose coupling between components.
- Reusability.

Disadvantages

- Incorrect extras may cause crashes.
- Implicit intents may fail if no matching app exists.
- Managing many intent keys can become difficult.

Frequently Asked Exam Questions

Short Questions (2–5 Marks)

- What is an Intent?
- Differentiate between Explicit and Implicit Intent.
- What is an Intent Filter?
- What is Bundle?
- Write any two Intent actions.

Long Questions (10–15 Marks)

- Explain Intent in Android with architecture.
- Differentiate between Explicit and Implicit Intent with examples.
- Explain how to pass data between Activities using Intent and Bundle.
- Explain Intent Filters with suitable examples.

MCQs

- Intent is used for? → Communication between components ✓
- Which intent specifies the target activity? → Explicit Intent ✓
- Which action opens a web page? → ACTION_VIEW ✓

- Which method adds data to an Intent? → putExtra() ✓
- Which object stores key-value pairs? → Bundle ✓

8. Android Project Structure (Complete Detailed Notes)

When you create a new Android project in **Android Studio**, many folders and files are automatically generated. Understanding the Android Project Structure is very important because every Android application is built using these folders.

This topic is frequently asked in:

- Theory Exams (5–15 Marks)
- Viva
- Practical Exams
- Interviews

What is Android Project Structure?

Definition

Android Project Structure refers to the organization of files and folders used in an Android application.

It contains:

- Source Code
- Layout Files
- Resources
- Images
- Manifest File
- Gradle Files
- Libraries

Android Project Structure Overview

MyApplication
|

```

├── manifests
│   └── AndroidManifest.xml
├── java
│   └── MainActivity.java
├── res
│   ├── drawable
│   ├── layout
│   ├── mipmap
│   ├── values
│   └── menu
├── Gradle Scripts
└── build.gradle

```

Complete Android Project Architecture

Android Project

```

|
├── Manifests
|   └── AndroidManifest.xml
|
├── Java / Kotlin
|   └── Source Code
|
├── Res (Resources)
|   ├── Layout
|   ├── Drawable
|   ├── Mipmap
|   ├── Values
|   ├── Menu
|   └── Raw
|
└── Gradle Scripts

```

1. manifests Folder

Contains:

`AndroidManifest.xml`

This is the most important file of every Android application.

AndroidManifest.xml

Purpose

It tells Android:

- App Name
 - Activities
 - Permissions
 - Services
 - Broadcast Receivers
 - Content Providers
-

Example

```
<manifest>

<application
    android:label="My App">

    <activity
        android:name=".MainActivity">

        <intent-filter>

            <action
                android:name=
                "android.intent.action.MAIN"/>

            <category
                android:name=
                "android.intent.category.LAUNCHER"/>

        </intent-filter>
```

```
</activity>

</application>

</manifest>
```

Real World Example

Suppose your app uses:

- Camera
- Internet
- GPS

Permissions are declared inside Manifest.

```
<uses-permission
android:name=
"android.permission.INTERNET"/>

<uses-permission
android:name=
"android.permission.CAMERA"/>
```

2. Java / Kotlin Folder

Contains application source code.

Example:

```
java
├─ com.example.myapp
│   │
│   └─ MainActivity.java
│
│   └─ LoginActivity.java
│
│   └─ DatabaseHelper.java
```

Purpose

Used for:

- Business Logic
 - Activity Code
 - Database Operations
 - API Calls
 - Event Handling
-

Example

```
public class MainActivity
extends AppCompatActivity {

    @Override

    protected void onCreate(
        Bundle savedInstanceState) {

        super.onCreate(
            savedInstanceState);

        setContentView(
            R.layout.activity_main);
    }
}
```

Real World Example

FoodPanda App

MainActivity.java

RestaurantActivity.java

CartActivity.java

PaymentActivity.java

Each screen has its own Java/Kotlin file.

3. res Folder (Resources)

The most important folder after Java.

Contains all non-code resources.

res

├─ drawable

├─ layout

├─ mipmap

├─ values

├─ menu

└─ raw

What is Resource?

Resource means:

- Images
 - Layouts
 - Colors
 - Strings
 - Icons
 - Menus
-

3.1 drawable Folder

Stores:

Images

Shapes

Backgrounds

Vector Graphics

Examples

logo.png

background.jpg

icon.xml

Real World Example

WhatsApp logo:

drawable/logo.png

Access Drawable

```
imageView.setImageResource(  
    R.drawable.logo);
```

3.2 layout Folder

Contains UI design XML files.

Example

activity_main.xml

activity_login.xml

activity_profile.xml

Sample Layout

```
<LinearLayout  
  
    android:layout_width=  
        "match_parent"  
  
    android:layout_height=  
        "match_parent">  
  
    <TextView  
  
        android:text=  
            "Hello Android"/>  
  
</LinearLayout>
```

Real World Example

Instagram:

Login Screen XML

Home Screen XML

Profile Screen XML

All stored in layout folder.

3.3 mipmap Folder

Stores application launcher icons.

Example:

mipmap-hdpi

mipmap-mdpi

mipmap-xhdpi

mipmap-xxhdpi

Why Multiple Icons?

Different Android devices have different screen densities.

Android automatically selects the appropriate icon.

Real World Example

App icon visible on home screen comes from mipmap folder.

3.4 values Folder

Contains configuration files.

Structure:

values

├─ strings.xml

├─ colors.xml

├─ themes.xml

└─ styles.xml

strings.xml

Stores text values.

Example:

```
<resources>
<string name="app_name">
My Application
</string>
</resources>
```

Usage

```
android:text="@string/app_name"
```

Advantages:

- Easy maintenance
 - Multi-language support
-

colors.xml

Stores colors.

```
<color name="red">
#FF0000
</color>
```

Usage:

```
android:textColor="@color/red"
```

themes.xml

Defines app theme.

Example:

```
Theme.Material3
```

Controls:

- Colors
 - Fonts
 - Appearance
-

Real World Example

Dark Mode support comes through themes.

3.5 menu Folder

Stores menu XML files.

Example:

main_menu.xml

Example Code

```
<menu>

<item

android:id="@+id/settings"

android:title="Settings"/>

</menu>
```

Real World Example

WhatsApp top-right menu:

Settings

New Group

Linked Devices

Stored in menu folder.

3.6 raw Folder

Stores raw files.

Examples:

music.mp3

video.mp4

pdf.pdf

Access

R.raw.music

Real World Example

Game background music.

4. Gradle Scripts

Gradle is Android's build system.

Purpose:

- Compile code
 - Manage dependencies
 - Generate APK
-

Structure

Gradle Scripts

|

|— build.gradle(Project)

|— build.gradle(Module)

|— settings.gradle

Project-Level build.gradle

Used for:

- Global configuration

Example:

```
plugins {  
  
id 'com.android.application'  
  
}
```

Module-Level build.gradle

Contains:

```
dependencies {  
  
    implementation  
    'androidx.appcompat:appcompat'  
  
}
```

Purpose:

Manage libraries.

Real World Example

If Retrofit library is required:

```
implementation  
'com.squareup.retrofit2:retrofit:2.9.0'
```

Build Process of Android App

Java/Kotlin Code

↓

XML Resources

↓

Gradle Build

↓

Compilation

↓

APK Generation

↓

Install on Device

What is APK?

Definition

APK = Android Package

It is the installable file format of Android applications.

Example:

WhatsApp.apk

Facebook.apk

APK Contents

APK

|

|— classes.dex

|— resources

|— manifest

|— assets

|— libraries

R File

Automatically generated file.

Purpose:

Connects Java code with resources.

Example:

```
R.layout.activity_main
```

```
R.id.button1
```

```
R.drawable.logo
```

Without R file:

Java code cannot access XML resources.

Real World Example

```
setContentView(  
    R.layout.activity_main);
```

Here R links Java and XML.

Android Project Flow

User Interface (XML)

↓

Java/Kotlin Code

↓

Manifest

↓

Gradle Build

↓

APK

↓

Android Device

Advantages of Proper Project Structure

- Easy maintenance
 - Better organization
 - Faster development
 - Team collaboration
 - Easy debugging
 - Resource management
-

Viva Questions

Q1: What is AndroidManifest.xml?

AndroidManifest.xml contains information about application components, permissions, and configurations.

Q2: What is the purpose of the layout folder?

It stores XML files used to design user interfaces.

Q3: What is the difference between drawable and mipmap?

- Drawable stores images and graphics.
 - Mipmap stores launcher icons.
-

Q4: What is Gradle?

Gradle is Android's build automation tool used to compile code and generate APKs.

Q5: What is APK?

APK (Android Package) is the installable file format of Android applications.

MCQs

1. Which file contains app permissions?

- A. strings.xml
 - B. colors.xml
 - C. AndroidManifest.xml ✓
 - D. styles.xml
-

2. Which folder stores XML layouts?

- A. drawable
 - B. values
 - C. layout ✓
 - D. mipmap
-

3. Which folder stores launcher icons?

- A. drawable
 - B. layout
 - C. mipmap ✓
 - D. values
-

4. What does APK stand for?

- A. Android Programming Kit
 - B. Android Package Kit ✓
 - C. Android Package Kernel
 - D. Application Package Key
-

5. Which file links resources with Java code?

- A. Manifest
- B. APK
- C. R File ✓
- D. Gradle

9. AndroidManifest.xml (Complete Detailed Notes)

AndroidManifest.xml is one of the most important files in every Android application. Without this file, Android cannot know:

- What the app is called
- Which Activity should start first
- What permissions the app needs
- What Services, Broadcast Receivers, and Content Providers exist
- What features the app supports

This topic is frequently asked in 5, 10, and 15 marks exams.

What is AndroidManifest.xml?

Definition

AndroidManifest.xml is a configuration file that describes the essential information about an Android application to the Android operating system.

Simple Words:

It is the identity card of an Android application.

Location of Manifest File

app

└─ manifests

└─ AndroidManifest.xml

Why is Manifest Important?

Android reads this file before running the app.

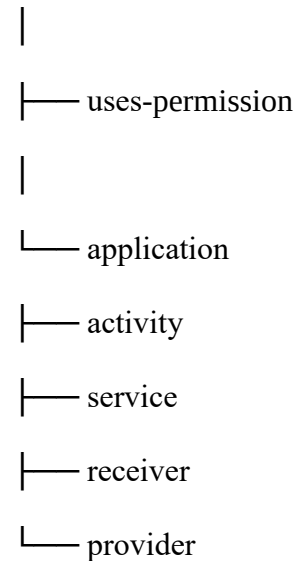
It tells Android:

- App package name
- Application name
- App icon
- Activities
- Services
- Broadcast Receivers
- Content Providers
- Permissions
- Intent Filters
- Minimum SDK version
- Target SDK version

Structure of AndroidManifest.xml

Hierarchy:

manifest



Complete Manifest Example

1. The <manifest> Tag

Purpose

It is the root element of the file.

Example

Important Attributes

Attribute	Purpose
<code>xmlns:android</code>	Android namespace
<code>package</code>	Unique app identifier

Package Name Example

It uniquely identifies the application on the Play Store.

Real-World Example

- `com.whatsapp`
- `com.instagram.android`
- `com.facebook.katana`

2. The <application> Tag

Purpose

Contains application-wide settings.

Example

Important Attributes

Attribute	Purpose
<code>android:label</code>	App name
<code>android:icon</code>	App icon
<code>android:theme</code>	App theme
<code>android:allowBackup</code>	Allow backup
<code>android:supportsRtl</code>	Right-to-left support

Real-World Example

When you install WhatsApp:

- App name appears.
- App icon appears.
- Theme is applied.

These settings come from the application tag.

3. The <activity> Tag

Purpose

Registers an Activity with Android.

Example

Multiple Activities

If an Activity is not declared here, Android usually cannot launch it.

Real-World Example

A shopping app may contain:

- HomeActivity
- ProductActivity
- CartActivity
- PaymentActivity

All must be registered in the Manifest.

4. Launch Activity (MAIN & LAUNCHER)

Purpose

Defines the first screen that opens when the app starts.

Example

Meaning

Item	Purpose
MAIN	Entry point of the app
LAUNCHER	Show icon in launcher

Flow

User taps App Icon

↓

Android reads Manifest



Finds MAIN + LAUNCHER



MainActivity starts

5. Intent Filters

Definition

Intent Filters declare which intents a component can handle.

Example

Real-World Example

When you click a web link:

- Browser declares an intent filter.
- Android knows the browser can handle `ACTION_VIEW`.

6. Permissions

Definition

Permissions allow access to protected features.

Internet Permission

Camera Permission

Location Permission

Storage Permission

Common Permissions

Permission	Purpose
INTERNET	Internet access
CAMERA	Use camera

Permission	Purpose
RECORD_AUDIO	Use microphone
ACCESS_FINE_LOCATION	Get GPS location
READ_CONTACTS	Read contacts
SEND_SMS	Send SMS

Real-World Example

Google Maps:

- Requests location permission.
- Without permission, GPS cannot be accessed.

7. Service Declaration

Example

Purpose:

- Background music
- Downloads
- Location tracking

Real-World Example

Spotify uses services for background playback.

8. Broadcast Receiver Declaration

Purpose:

- Listen for system events.
- React automatically.

Real-World Example

Battery Saver apps receive `BATTERY_LOW` broadcasts.

9. Content Provider Declaration

Purpose:

- Share app data.
- Provide CRUD operations.

Real-World Example

Contacts app exposes contacts through a Content Provider.

10. SDK Configuration

Usually defined in Gradle, but conceptually important.

- minSdk: Minimum Android version supported.
- targetSdk: Android version optimized for.
- compileSdk: SDK used to compile the app.

Example

Complete Real-World Manifest Example

Food Delivery App:

Advantages of Manifest File

- Central configuration file.
- Registers all app components.
- Declares permissions.
- Controls app behavior.
- Enables inter-component communication.
- Required for every Android app.

Disadvantages

- Wrong configuration may crash the app.
- Missing permissions cause functionality failure.
- Incorrect component declarations lead to runtime errors.

Frequently Asked Exam Questions

Short Questions (2–5 Marks)

- What is AndroidManifest.xml?
- What is the purpose of the package attribute?
- Write the Internet permission.
- What is an intent filter?
- What is the role of the application tag?

Long Questions (10–15 Marks)

- Explain AndroidManifest.xml with a neat diagram.
- Explain the structure of Manifest and its major tags.
- Discuss permissions, activities, services, receivers, and providers declared in Manifest.
- Explain MAIN and LAUNCHER intent filters with examples.

MCQs

- Which file contains app permissions? → AndroidManifest.xml ✓
- Which tag defines the launch activity? → intent-filter with MAIN and LAUNCHER ✓
- Which attribute specifies the app icon? → android:icon ✓
- Which permission allows internet access? → android.permission.INTERNET ✓
- Which tag registers a background service? → <service> ✓

Quick Revision (5 Minutes Before Exam)

- Manifest: Configuration file of Android app.
- <manifest>: Root tag.
- <application>: App-wide settings.
- <activity>: Registers activities.
- <service>: Registers services.
- <receiver>: Registers broadcast receivers.
- <provider>: Registers content providers.
- uses-permission: Declares permissions.
- MAIN + LAUNCHER: Defines the first screen of the app.

10. Layouts in Android (Complete Detailed Notes)

Layouts are one of the most important topics in Android Development. Every Android application screen is designed using layouts. This topic is frequently asked in 5, 10, and 15 marks exams.

What is a Layout?

Definition

A Layout is a container that defines the structure and arrangement of UI components on the screen.

Simple Words:

Layout decides where buttons, text fields, images, and other widgets will appear on the screen.

Example:

In a Login Screen:

- Username field
- Password field
- Login button

All these widgets are arranged using a Layout.

View and ViewGroup

1. View

A View is a single UI component.

Examples:

- TextView
- Button
- EditText
- ImageView

Example:

Here, Button is a View.

2. ViewGroup

A ViewGroup is a container that holds multiple Views.

Examples:

- LinearLayout
- RelativeLayout
- ConstraintLayout
- FrameLayout
- TableLayout

Example:

Here, LinearLayout is a ViewGroup.

Hierarchy

ViewGroup (Layout)

|

|— View (Button)

|— View (TextView)

|— View (ImageView)

Types of Layouts

Android provides several layouts:

- LinearLayout
- RelativeLayout
- ConstraintLayout
- FrameLayout
- TableLayout

1. LinearLayout

Definition

Arranges views in a single row or single column.

Vertical LinearLayout

Output:

[Username]

[Password]

[Login]

Horizontal LinearLayout

Output:

[Yes] [No]

Important Attributes

Attribute	Purpose
<code>android:orientation</code>	Vertical or horizontal
<code>android:gravity</code>	Align children
<code>android:layout_weight</code>	Distribute space

Weight Example

Real-World Example:

- Login Screen
- Registration Form
- Settings Screen

Advantages

- Simple to use
- Easy arrangement
- Good for forms

Disadvantages:

- Deep nesting reduces performance.
- Complex UI becomes difficult.

2. RelativeLayout

Definition

Positions views relative to each other or the parent.

Example

Output:

Welcome

[Login]

Important Attributes

Attribute	Purpose
<code>layout_below</code>	Place below another view
<code>layout_above</code>	Place above another view
<code>layout_toRightOf</code>	Place to the right
<code>layout_toLeftOf</code>	Place to the left
<code>layout_centerInParent</code>	Center in parent

Real-World Example

Old Android apps commonly used RelativeLayout for:

- Profile screens
- Login pages
- Dashboard screens

Advantages

- Less nesting than LinearLayout.
- Flexible positioning.

Disadvantages:

- Complex rules become hard to manage.
- Largely replaced by ConstraintLayout.

3. ConstraintLayout ★ (Most Important)

Definition

A modern layout that positions views using constraints.

Google recommends using ConstraintLayout for most screens.

Example

Output:

Button appears in the center.

How Constraints Work

Parent

|

|— Top Constraint

|— Bottom Constraint

|— Start Constraint

|— End Constraint

Important Constraints

Constraint	Purpose
Top_toTopOf	Connect top to top
Bottom_toBottomOf	Connect bottom to bottom
Start_toStartOf	Connect start to start
End_toEndOf	Connect end to end
Top_toBottomOf	Place below another view

Real-World Example

Modern apps such as:

- Gmail
- Google Maps
- YouTube
- Instagram

use `ConstraintLayout` extensively.

Advantages

- Excellent performance
- Flat hierarchy
- Flexible UI
- Responsive design

Disadvantages:

- Initially harder to learn.
- Complex constraints can be confusing.

4. `FrameLayout`

Definition

Displays views on top of each other.

Example

Output:

- Background image
- Text shown on top

Real-World Example

- Fragment container
- Splash screen
- Image with overlay text

Advantages

- Very simple
- Good for overlapping views
- Perfect for fragments

Disadvantages:

- Not suitable for complex arrangements.

5. TableLayout

Definition

Arranges views in rows and columns.

Example

Output:

Name [__]

Age [__]

Real-World Example

- Timetable
- Marks Sheet
- Invoice

Advantages

- Good tabular arrangement.
- Easy row-column structure.

Disadvantages:

- Less flexible.
- Not commonly used in modern apps.

Nested Layouts

Layouts can contain other layouts.

Example:

Output:

Profile

[Edit] [Delete]

Problem with Deep Nesting

Too many nested layouts:

- Consume more memory.
- Reduce performance.
- Increase rendering time.

Solution:

- Use ConstraintLayout.

Common Layout Attributes

Attribute	Purpose
<code>layout_width</code>	Width of view
<code>layout_height</code>	Height of view
<code>layout_margin</code>	Outer spacing
<code>padding</code>	Inner spacing
<code>gravity</code>	Align content
<code>layout_gravity</code>	Align view in parent
<code>id</code>	Unique identifier

Example

Which Layout Should You Use?

Layout	Best Use
LinearLayout	Simple rows/columns
RelativeLayout	Older relative positioning
ConstraintLayout	Most modern screens ★
FrameLayout	Fragments/overlays

Layout	Best Use
TableLayout	Tabular data

Viva Questions

- What is a Layout? → A container that arranges UI components.
- Difference between View and ViewGroup? → View is a widget; ViewGroup is a container.
- Which layout is recommended by Google? → ConstraintLayout.
- Which layout arranges views vertically or horizontally? → LinearLayout.
- Which layout stacks views on top of each other? → FrameLayout.
- Which layout is used for rows and columns? → TableLayout.

MCQs

- A layout is a: → ViewGroup ✓
- Which layout is best for modern UI? → ConstraintLayout ✓
- Which layout arranges views in a single row or column? → LinearLayout ✓
- Which layout stacks views? → FrameLayout ✓
- Which layout is used for tabular data? → TableLayout ✓

Important Exam Question (15 Marks)

Explain Android Layouts in detail. Discuss View and ViewGroup, LinearLayout, RelativeLayout, ConstraintLayout, FrameLayout, and TableLayout with suitable examples and diagrams.

1. Widgets in Android (Complete Detailed Notes)

What are Widgets?

Definition

Widgets are UI (User Interface) components that allow users to interact with an Android application.

Simple Words

Widgets are the controls visible on the screen such as:

- Button

- TextView
- EditText
- ImageView
- CheckBox
- RadioButton

Without widgets, users cannot interact with an app.

Real World Example

Login Screen

```
-----  
  
Username  
  
[ _____ ]  
  
Password  
  
[ _____ ]  
  
[ Login ]  
  
-----
```

Widgets used:

- TextView
- EditText
- Button

Widget Hierarchy

Layout (ViewGroup)

```
|  
|  
├─ TextView  
├─ EditText  
└─ Button
```


└─ ImageView
└─ CheckBox

1. TextView

Definition

TextView displays text on the screen.

Real World Example

Welcome to Facebook

"Welcome to Facebook" is shown using TextView.

XML Example

```
<TextView
    android:id="@+id/txtName"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello Android"
    android:textSize="24sp"/>
```

Output

Hello Android

Java Example

```
TextView tv;

tv=findViewById(R.id.txtName);

tv.setText("Welcome");
```

Important Attributes

Attribute	Purpose
text	Display text
textColor	Text color
textSize	Font size
gravity	Alignment
maxLines	Maximum lines

2. EditText

Definition

EditText allows users to enter data.

Real World Example

Username:

[_____]

User types data.

XML Example

```
<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Name"/>
```

Output

Enter Name

Java Example

```
EditText et;
```

```
et=findViewById(R.id.etName);  
  
String name=et.getText().toString();
```

Important Attributes

Attribute	Purpose
hint	Placeholder
inputType	Type of input
maxLength	Maximum length

Input Types

```
android:inputType="text"  
android:inputType="number"  
android:inputType="phone"  
android:inputType="textPassword"
```

3. Button

Definition

A Button performs an action when clicked.

Real World Example

```
[ LOGIN ]
```

XML Example

```
<Button  
    android:id="@+id/btnLogin"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Login"/>
```

Java Example

```
Button btn;

btn=findViewById(R.id.btnLogin);

btn.setOnClickListener(new View.OnClickListener() {

    @Override
    public void onClick(View v) {

        Toast.makeText(
            MainActivity.this,
            "Login Clicked",
            Toast.LENGTH_SHORT).show();

    }

});
```

Output

Login Clicked

4. ImageView

Definition

ImageView displays images.

Real World Example

Instagram profile picture.

XML Example

```
<ImageView
    android:id="@+id/img"
    android:layout_width="150dp"
    android:layout_height="150dp"
    android:src="@drawable/logo"/>
```

Java Example

```
ImageView img;  
  
img=findViewById(R.id.img);  
  
img.setImageResource(  
R.drawable.logo);
```

Output

Logo image displayed.

5. CheckBox

Definition

CheckBox allows multiple selections.

Real World Example

Registration Form:

- ☒ Cricket
- ☒ Football
- ☐ Hockey

Multiple options can be selected.

XML Example

```
<CheckBox  
    android:id="@+id/chkJava"  
    android:text="Java"/>
```

Java Example

```
CheckBox chk;  
  
chk=findViewById(R.id.chkJava);  
  
if(chk.isChecked())  
{  
    // Selected  
}
```

6. RadioButton

Definition

RadioButton allows only one option selection.

Real World Example

Gender Selection:

```
(•) Male  
( ) Female  
( ) Other
```

Only one can be selected.

XML Example

```
<RadioButton  
    android:id="@+id/rbMale"  
    android:text="Male"/>
```

RadioGroup

RadioButtons are placed inside RadioGroup.

Example

```
<RadioGroup

android:layout_width="wrap_content"
android:layout_height="wrap_content">

<RadioButton
    android:text="Male"/>

<RadioButton
    android:text="Female"/>

</RadioGroup>
```

7. Spinner

Definition

Spinner displays a dropdown list.

Real World Example

Country Selection:

Pakistan ▼

XML Example

```
<Spinner
    android:id="@+id/spCountry"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>
```

Java Example

```
String countries[]={
    "Pakistan",
    "India",
    "China"
```

```
};

ArrayAdapter adapter=
new ArrayAdapter(
this,
android.R.layout.simple_spinner_item,
countries);

spinner.setAdapter(adapter);
```

Output

Pakistan

India

China

8. AutoCompleteTextView

Definition

Suggests values while typing.

Real World Example

Google Search Suggestions.

XML Example

```
<AutoCompleteTextView
    android:id="@+id/autoCity"
    android:hint="Enter City"/>
```

Java Example

```
String cities[]={
"Lahore",
"Karachi",
"Islamabad"
```

```
};

ArrayAdapter adapter=
new ArrayAdapter(
this,
android.R.layout.simple_list_item_1,
cities);

autoCity.setAdapter(adapter);
```

Output

Type: L

Suggestions:

Lahore

9. Switch

Definition

Switch is used to turn a setting ON or OFF.

Real World Example

Dark Mode

ON / OFF

XML Example

```
<Switch
    android:id="@+id/swDark"
    android:text="Dark Mode"/>
```

Java Example

```
if (swDark.isChecked())
{
    // ON
```

```
}  
else  
{  
    // OFF  
}
```

10. ProgressBar

Definition

Shows progress of an operation.

Real World Example

File Download.

Downloading...



XML Example

```
<ProgressBar  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"/>
```

Horizontal ProgressBar

```
<ProgressBar  
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
style="?android:attr/progressBarStyleHorizontal"/>
```

11. ListView

Definition

Displays a scrollable list of items.

Real World Example

Contact List.

Ali

Ahmed

Sara

Ayesha

XML Example

```
<ListView
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

Java Example

```
String names[]={
    "Ali",
    "Ahmed",
    "Sara"
};

ArrayAdapter adapter=
new ArrayAdapter(
    this,
    android.R.layout.simple_list_item_1,
    names);

listView.setAdapter(adapter);
```

Output

Ali

Ahmed

Sara

12. RecyclerView ★ MOST IMPORTANT

Definition

RecyclerView is an advanced version of ListView used to display large datasets efficiently.

Why RecyclerView?

ListView creates many views.

RecyclerView reuses old views.

Result:

- Faster
 - Better Performance
 - Less Memory Usage
-

Real World Example

Instagram Feed

Post 1

Post 2

Post 3

...

1000 Posts

RecyclerView handles all efficiently.

Components of RecyclerView

RecyclerView

|

└─ Adapter
└─ ViewHolder
└─ LayoutManager

RecyclerView XML

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recyclerView"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"/>
```

Layout Manager

```
recyclerView.setLayoutManager(  
    new LinearLayoutManager(this));
```

List View vs RecyclerView

List View	RecyclerView
Older	Modern
Slower	Faster
Less flexible	Highly flexible
No ViewHolder by default	Uses ViewHolder
Less efficient	More efficient

Most Used Widgets in Real Apps

App	Widgets Used
WhatsApp	TextView, RecyclerView, ImageView
Facebook	RecyclerView, Button
Instagram	RecyclerView, ImageView
FoodPanda	RecyclerView, Spinner
Gmail	RecyclerView, TextView

Viva Questions

What is a Widget?

A widget is a UI component used for user interaction.

Difference between TextView and EditText?

- TextView displays text.
- EditText receives input.

Difference between CheckBox and RadioButton?

- CheckBox = Multiple selections.
- RadioButton = Single selection.

What is Spinner?

A dropdown list widget.

Why RecyclerView is preferred?

Because it is faster and memory efficient.

MCQs

Which widget accepts user input?

✓ EditText

Which widget shows images?

✓ ImageView

Which widget supports multiple selections?

✓ CheckBox

Which widget shows dropdown options?

✓ Spinner

Which widget is best for large lists?

✓ RecyclerView

Important Exam Question (15 Marks)

Explain Android Widgets in detail. Discuss TextView, EditText, Button, ImageView, CheckBox, RadioButton, Spinner, AutoCompleteTextView, Switch, ProgressBar, ListView, and RecyclerView with suitable examples and code snippets.

12. Event Handling in Android (Complete Detailed Notes)

Event Handling is one of the most important concepts in Android because Android applications are **event-driven**.

This means:

☞ User performs an action

↓

☞ Event occurs

↓

☞ Android responds

What is an Event?

Definition

An **Event** is an action performed by the user or system.

Examples:

- Clicking a button
 - Touching screen
 - Typing text
 - Long pressing
 - Scrolling list
 - Receiving notification
-

Real World Example

When user clicks:

[LOGIN]

Button Click Event occurs.

Android executes code.

What is Event Handling?

Definition

Event Handling is the process of detecting and responding to user actions.

Flow

User Action

↓

Event Generated

↓

Listener Detects Event

↓

Handler Executes Code

↓

Result Displayed

Event Listener

Definition

A Listener is an interface that listens for events.

Examples:

- OnClickListener
 - OnLongClickListener
 - onTouchListener
 - OnKeyListener
 - onFocusChangeListener
-

Event Handling Architecture

User

↓

Widget

↓

Listener

↓

Handler Code

↓

Output

1. OnClickListener ★ MOST IMPORTANT

Definition

Used when user clicks a widget.

Most commonly used event in Android.

Real World Example

[LOGIN]

User clicks Login button.

XML

```
<Button
    android:id="@+id/btnLogin"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Login"/>
```

Java Code

```
Button btnLogin;

btnLogin=findViewById(R.id.btnLogin);

btnLogin.setOnClickListener(
    new View.OnClickListener() {

        @Override
        public void onClick(View v) {

            Toast.makeText(
                MainActivity.this,
                "Login Successful",
                Toast.LENGTH_SHORT).show();

        }
    });
```

Output

Login Successful

Example 2

Change TextView Text

XML

```
<TextView
    android:id="@+id/txtMessage"
    android:text="Hello"/>
```

Java

```
TextView txt;
Button btn;

txt=findViewById(R.id.txtMessage);
btn=findViewById(R.id.btnLogin);

btn.setOnClickListener(v -> {

    txt.setText("Welcome");

});
```

Output

Before:

Hello

After Button Click:

Welcome

2. Long Click Event

Definition

Occurs when user presses and holds a view.

Real World Example

WhatsApp Chat

Press and Hold Message

Options appear:

Delete

Forward

Copy

Java Example

```
btnLogin.setOnLongClickListener(  
    new View.OnLongClickListener() {  
  
        @Override  
  
        public boolean onLongClick(View v) {  
  
            Toast.makeText(  
                MainActivity.this,  
                "Long Click",  
                Toast.LENGTH_SHORT).show();  
  
            return true;  
        }  
    });
```

Output

Long Click

Difference

Click	Long Click
Quick tap	Press and hold
Faster	Longer action
Common	Less frequent

3. Touch Event

Definition

Triggered when user touches the screen.

Types

Event	Meaning
ACTION_DOWN	Finger touches screen
ACTION_MOVE	Finger moves
ACTION_UP	Finger released

Real World Example

Drawing Apps

Games

Maps

Java Example

```
imageView.setOnTouchListener(  
    new View.OnTouchListener() {  
        @Override  
        public boolean onTouch(  
            View v,  
            MotionEvent event) {
```

```
switch(event.getAction()) {  
  
    case MotionEvent.ACTION_DOWN:  
  
        Toast.makeText(  
            MainActivity.this,  
            "Touched",  
            Toast.LENGTH_SHORT).show();  
  
        break;  
  
    }  
  
    return true;  
  
    }  
  
});
```

Output

Touched

Touch Event Flow

Finger Touch

↓

ACTION_DOWN

↓

ACTION_MOVE

↓

ACTION_UP

4. Key Event

Definition

Occurs when keyboard keys are pressed.

Real World Example

Search Box

User presses:

ENTER

Search starts.

Java Example

```
editText.setOnKeyListener(  
    new View.OnKeyListener() {  
        @Override  
        public boolean onKey(  
            View v,  
            int keyCode,  
            KeyEvent event) {  
            if (keyCode ==  
                KeyEvent.KEYCODE_ENTER) {  
                Toast.makeText(  
                    MainActivity.this,  
                    "Enter Pressed",  
                    Toast.LENGTH_SHORT).show();  
            }  
            return false;  
        }  
    });
```

Output

Enter Pressed

5. Focus Event

Definition

Occurs when a widget gains or loses focus.

Real World Example

Login Form

Username Field

↓

Focused

↓

Highlighted

Java Example

```
editText.setOnFocusChangeListener(  
    new View.OnFocusChangeListener() {  
        @Override  
        public void onFocusChange(  
            View v,  
            boolean hasFocus) {  
            if (hasFocus) {  
                Toast.makeText(  
                    MainActivity.this,  
                    "Focused",  
                    Toast.LENGTH_SHORT).show();  
            }  
        }  
    });
```

Output

Focused

6. CheckBox Event

XML

```
<CheckBox  
android:id="@+id/chkJava"  
android:text="Java"/>
```

Java

```
CheckBox chk;  
  
chk=findViewById(R.id.chkJava);  
  
chk.setOnCheckedChangeListener(  
    (buttonView, isChecked) -> {  
  
        if (isChecked) {  
  
            Toast.makeText(  
                this,  
                "Checked",  
                Toast.LENGTH_SHORT).show();  
  
        }  
  
    });
```

Output

Checked

7. RadioButton Event

Example

```
RadioGroup rg;
```

```
int selectedId=
rg.getCheckedRadioButtonId();

RadioButton rb=
findViewById(selectedId);

String value=
rb.getText().toString();
```

Output

Male

or

Female

8. Spinner Event

Java Example

```
spinner.setOnItemClickListener(
new AdapterView.OnItemClickListener() {
@Override
public void onItemClick(
AdapterView<?> parent,
View view,
int position,
long id) {
String country=
parent.getItemAtPosition(
position).toString();
Toast.makeText(
MainActivity.this,
country,
Toast.LENGTH_SHORT).show();
}
```

```
@Override

public void onNothingSelected(
AdapterView<?> parent) {

}

});
```

Output

Pakistan

9. RecyclerView Click Event ★

Very Important Practical Question

Real World Example

Instagram Feed

User clicks:

Post 1



Post Detail Opens

Adapter Code

```
holder.itemView.setOnClickListener(v -> {

Toast.makeText(
context,
list.get(position),
Toast.LENGTH_SHORT).show();

});
```

Output

Ali

clicked

Anonymous Inner Class Method

Old Android Style

```
btn.setOnClickListener(  
    new View.OnClickListener() {  
  
        @Override  
  
        public void onClick(View v) {  
  
        }  
  
    });
```

Lambda Expression Method

Modern Android

```
btn.setOnClickListener(v -> {  
  
});
```

Event Handling Example

Login Form

XML

```
<EditText  
    android:id="@+id/etUser"/>  
  
<Button  
    android:id="@+id/btnLogin"  
    android:text="Login"/>
```

Java

```
EditText etUser;  
Button btnLogin;  
  
etUser=findViewById(R.id.etUser);  
btnLogin=findViewById(R.id.btnLogin);  
  
btnLogin.setOnClickListener(v -> {  
  
    String name=  
etUser.getText().toString();  
  
    Toast.makeText(  
this,  
"Welcome "+name,  
Toast.LENGTH_SHORT).show();  
  
});
```

Output

Input:

Ali

Output:

Welcome Ali

Common Android Events

Event	Listener
Click	OnClickListener
Long Click	OnLongClickListener
Touch	OnTouchListener
Key Press	OnKeyListener
Focus Change	OnFocusChangeListener
Item Selected	OnItemSelectedListener

Real World Apps

WhatsApp

- Click Chat
 - Long Press Message
 - Touch Emoji
-

Instagram

- Click Post
 - Double Tap Like
 - Scroll Feed
-

FoodPanda

- Select Category
 - Click Restaurant
 - Add to Cart
-

Advantages of Event Handling

- Interactive UI
 - Better user experience
 - Dynamic applications
 - Easy communication with users
-

Viva Questions

What is Event Handling?

Responding to user actions.

What is OnClickListener?

Used to handle button click events.

Difference between Click and Long Click?

- Click = Tap
- Long Click = Press and Hold

Which listener handles touch events?

OnTouchListener

Which listener handles Spinner selection?

OnItemSelectedListener

MCQs

Which listener handles button click?

✓ OnClickListener

Which listener handles long press?

✓ OnLongClickListener

Which event occurs when finger touches screen?

✓ ACTION_DOWN

Which listener handles touch events?

✓ OnTouchListener

Which listener handles focus changes?

✓ OnFocusChangeListener

Important Exam Question (15 Marks)

Explain Event Handling in Android. Discuss OnClickListener, OnLongClickListener, OnTouchListener, OnKeyListener, and OnFocusChangeListener with suitable examples and code snippets.

3. RecyclerView + Adapter + ViewHolder (Complete Detailed Notes)

★ **Most Important Topic for Practical + Theory Exam**

Almost every modern Android application uses RecyclerView.

Examples:

- WhatsApp Chat List
- Instagram Feed
- Facebook Posts
- Gmail Emails
- FoodPanda Restaurants
- Daraz Product List

What is RecyclerView?

Definition

RecyclerView is a ViewGroup used to display a large collection of data efficiently.

It displays data in:

- List Form

- Grid Form
 - Horizontal List
 - Vertical List
-

Simple Words

RecyclerView shows large amounts of data while reusing old views to save memory.

Real World Example

WhatsApp Chat List

Ali

Ahmed

Sara

Ayesha

Hamza

...

1000 Chats

RecyclerView loads only visible items.

As user scrolls:

- Old rows are reused.
- New data is displayed.

This is called Recycling.

Why RecyclerView?

Before RecyclerView, Android used:

ListView

Problems:

- Slow performance
- High memory usage
- Limited customization

Google introduced:

RecyclerView

to solve these problems.

RecyclerView Working

Without RecyclerView:

1000 Items

↓

1000 Views Created

↓

High Memory Usage

With RecyclerView:

1000 Items

↓

Only 10-15 Views Created

↓

Views Reused

↓

Better Performance

RecyclerView Architecture

RecyclerView

|
├ Adapter
├ ViewHolder
└ LayoutManager

All three are required.

Components of RecyclerView

1. RecyclerView

Displays data.

2. Adapter

Provides data to RecyclerView.

3. ViewHolder

Stores row views.

4. LayoutManager

Controls item arrangement.

Real World Example

Instagram Feed

RecyclerView

↓
Adapter
↓
Post Data
↓
ViewHolder
↓
Post Layout
↓
Display on Screen

Step 1: Add RecyclerView Dependency

Usually already available.

```
implementation 'androidx.recyclerview:recyclerview:1.3.2'
```

Step 2: Add RecyclerView in XML

activity_main.xml

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recyclerView"  
  
    android:layout_width="match_parent"  
  
    android:layout_height="match_parent"/>
```

Output

Empty RecyclerView appears.

No data yet.

Step 3: Create Row Layout

Every item needs its own design.

item_student.xml

```
<LinearLayout  
  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:padding="15dp"  
    android:orientation="vertical">  
  
    <TextView  
  
        android:id="@+id/txtName"  
        android:textSize="20sp"  
        android:text="Student Name"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"/>  
  
</LinearLayout>
```

Output

Ali

Ahmed

Sara

Each row uses this layout.

Step 4: Create Model Class

Model stores data.

Student.java

```
public class Student {  
    String name;  
    public Student(String name) {  
        this.name=name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

Why Model Class?

Stores data cleanly.

Example:

```
new Student("Ali");  
new Student("Sara");
```

Step 5: Create Adapter

Most important part.

StudentAdapter.java

```
public class StudentAdapter  
extends RecyclerView.Adapter<  
StudentAdapter.ViewHolder> {  
    ArrayList<Student> list;  
    public StudentAdapter(  
        ArrayList<Student> list) {
```

```
        this.list=list;
    }
}
```

Adapter Responsibilities

Adapter:

- Creates row
 - Binds data
 - Returns item count
-

Adapter Methods

RecyclerView Adapter contains:

```
onCreateViewHolder()
onBindViewHolder()
getItemCount()
```

1. onCreateViewHolder()

Purpose

Creates row layout.

```
@Override
public ViewHolder onCreateViewHolder(
    ViewGroup parent,
    int viewType) {
    View view=
    LayoutInflater.from(
    parent.getContext())
```

```
.inflate(  
R.layout.item_student,  
parent,  
false);  
  
return new ViewHolder(view);  
}
```

Real World Example

Suppose:

Ali

Ahmed

Sara

RecyclerView creates row design using:

```
onCreateViewHolder()
```

2. onBindViewHolder()

Purpose

Places data inside views.

```
@Override  
  
public void onBindViewHolder(  
ViewHolder holder,  
int position) {  
holder.txtName.setText(  
list.get(position).getName());  
}  

```

Example

Position 0

Ali

Position 1

Ahmed

Position 2

Sara

3. getItemCount()

Purpose

Returns total rows.

```
@Override  
  
public int getItemCount() {  
    return list.size();  
}
```

Example:

```
list.size() = 3
```

RecyclerView shows:

Ali

Ahmed

Sara

ViewHolder

Definition

ViewHolder stores references of row views.

Why ViewHolder?

Without ViewHolder:

```
Every Scroll  
  
↓  
  
findViewById()  
  
↓  
  
Slow
```

With ViewHolder:

```
Views Stored Once  
  
↓  
  
Reused  
  
↓  
  
Fast
```

Create ViewHolder

```
public class ViewHolder  
  
extends RecyclerView.ViewHolder {  
  
    TextView txtName;  
  
    public ViewHolder(View itemView) {  
  
        super(itemView);  
  
        txtName=  
        itemView.findViewById(  
        R.id.txtName);  
    }  
}
```

```
}  
}
```

Complete Adapter

```
public class StudentAdapter  
  
extends RecyclerView.Adapter<  
StudentAdapter.ViewHolder> {  
  
    ArrayList<Student> list;  
  
    public StudentAdapter(  
        ArrayList<Student> list) {  
  
        this.list=list;  
  
    }  
  
    @Override  
  
    public ViewHolder onCreateViewHolder(  
        ViewGroup parent,  
        int viewType) {  
  
        View view=  
  
        LayoutInflater.from(  
            parent.getContext())  
  
            .inflate(  
                R.layout.item_student,  
                parent,  
                false);  
  
        return new ViewHolder(view);  
  
    }  
  
    @Override  
  
    public void onBindViewHolder(  
        ViewHolder holder,  
        int position) {  
  
        holder.txtName.setText(  
            list.get(position).getName());  
  
    }  
  
    @Override  
  
    public int getItemCount() {
```

```
return list.size();

}

public class ViewHolder
extends RecyclerView.ViewHolder {

    TextView txtName;

    public ViewHolder(View itemView) {

        super(itemView);

        txtName=
        itemView.findViewById(
        R.id.txtName);

    }

}

}
```

Step 6: Use RecyclerView in Activity

MainActivity.java

```
RecyclerView recyclerView;

ArrayList<Student> list;

StudentAdapter adapter;

recyclerView=
findViewById(R.id.recyclerView);

list=new ArrayList<>();

list.add(new Student("Ali"));
list.add(new Student("Ahmed"));
list.add(new Student("Sara"));

adapter=
new StudentAdapter(list);

recyclerView.setAdapter(adapter);

recyclerView.setLayoutManager(
new LinearLayoutManager(this));
```

Output

Ali

Ahmed

Sara

Layout Managers

LayoutManager controls display style.

1. LinearLayoutManager

Vertical list.

```
recyclerView.setLayoutManager(  
    new LinearLayoutManager(this);
```

Output:

Ali

Ahmed

Sara

2. Horizontal Layout

```
new LinearLayoutManager(  
    this,  
    LinearLayoutManager.HORIZONTAL,  
    false)
```

Output:

Ali Ahmed Sara

3. GridLayoutManager

```
recyclerView.setLayoutManager(  
  
new GridLayoutManager(  
this,  
2));
```

Output:

```
Ali      Ahmed  
Sara     Hamza
```

4. StaggeredGridLayoutManager

Pinterest Style.

Different Heights

Different Sizes

RecyclerView Click Event

Most Asked Practical Question

Adapter Click

```
holder.itemView.setOnClickListener(v -> {  
  
Toast.makeText(  
  
v.getContext(),  
  
list.get(position).getName(),  
  
Toast.LENGTH_SHORT).show();  
  
});
```

Output

Click:

```
Ali
```

Shows:

Ali

RecyclerView Flow

Data

↓

Adapter

↓

ViewHolder

↓

RecyclerView

↓

User

Real World Examples

WhatsApp

Chat List

↓

RecyclerView

Instagram

Posts

↓

RecyclerView

Facebook

News Feed



RecyclerView

Daraz

Products



RecyclerView

ListView vs RecyclerView

ListView	RecyclerView
Old	Modern
Slow	Fast
Less Flexible	More Flexible
No ViewHolder Pattern	ViewHolder Built-in
Less Efficient	Highly Efficient

Advantages of RecyclerView

- High performance
 - Reuses views
 - Less memory usage
 - Supports animation
 - Flexible layouts
 - Better user experience
-

Disadvantages

- More code than ListView
 - Slightly difficult for beginners
-

Viva Questions

What is RecyclerView?

A ViewGroup used to display large datasets efficiently.

Why RecyclerView is faster?

Because it recycles old views.

What is Adapter?

Adapter provides data to RecyclerView.

What is ViewHolder?

Stores row views for reuse.

What is LayoutManager?

Controls item arrangement.

MCQs

RecyclerView is used for?

✓ Displaying large data efficiently

Which class provides data?

✓ Adapter

Which class stores row views?

✓ ViewHolder

Which LayoutManager displays grid?

✓ GridLayoutManager

Why RecyclerView is faster?

✓ View Recycling

Important Exam Question (15 Marks)

Explain RecyclerView in Android. Discuss Adapter, ViewHolder, LayoutManager, working mechanism, advantages over ListView, and provide a complete example with code.

4. SQLite Database in Android (Complete Detailed Notes)

★ **Most Important Topic for Practical + Theory Exam**

Almost every Android application stores data.

Examples:

- WhatsApp → Messages
- Facebook → User Data
- FoodPanda → Orders
- Daraz → Products
- Notes App → Notes

This data is stored using a database.

What is SQLite?

Definition

SQLite is a lightweight relational database built into Android.

It stores data in tables.

No separate server is required.

Simple Words

SQLite is Android's built-in database used to save app data permanently.

Real World Example

Notes App

User creates note:

Buy Milk

App closes.

After reopening:

Buy Milk

still exists.

Because it was saved in SQLite Database.

Why SQLite?

Without SQLite:

App Closed

↓

Data Lost

With SQLite:

App Closed

↓

Data Saved

↓

Data Retrieved Later

SQLite Architecture

Android App

↓

SQLite Database

↓

Tables

↓

Rows & Columns

Example Database

Student Table

ID	Name	Age
----	------	-----

1	Ali	20
---	-----	----

2	Sara	22
---	------	----

3	Ahmed	21
---	-------	----

CRUD Operations

SQLite mainly performs:

C → Create

Insert Data

R → Read

Retrieve Data

U → Update

Modify Data

D → Delete

Remove Data

SQLiteOpenHelper

Definition

SQLiteOpenHelper is a helper class used to create and manage databases.

Why Use SQLiteOpenHelper?

Instead of manually creating databases:

Android provides:

`SQLiteOpenHelper`

which manages:

- Database Creation
 - Database Upgrades
 - Table Management
-

Create Database Helper Class

DatabaseHelper.java

```
public class DatabaseHelper
extends SQLiteOpenHelper {

    public DatabaseHelper(Context context) {

        super(
            context,
            "StudentDB",
            null,
            1);

    }

    @Override
    public void onCreate(SQLiteDatabase db) {

    }

    @Override
    public void onUpgrade(
        SQLiteDatabase db,
        int oldVersion,
        int newVersion) {

    }

}
```

Constructor Explanation

```
super(
context,
"StudentDB",
null,
1);
```

Parameter	Meaning
context	Current activity
StudentDB	Database Name
null	Cursor Factory
1	Database Version

onCreate()

Purpose

Executed only once.

Used for creating tables.

Create Table

```
@Override

public void onCreate(
    SQLiteDatabase db) {

    db.execSQL(

        "CREATE TABLE Student(" +

        "id INTEGER PRIMARY KEY AUTOINCREMENT," +

        "name TEXT," +

        "age INTEGER)");

}
```

Table Structure

Student

ID

NAME

AGE

onUpgrade()

Used when database version changes.

Example:

```
@Override

public void onUpgrade(
    SQLiteDatabase db,
    int oldVersion,
    int newVersion) {

    db.execSQL(
        "DROP TABLE IF EXISTS Student");

    onCreate(db);

}
```

Complete Database Flow

App Starts

↓

DatabaseHelper

↓

onCreate()

↓

Table Created

↓

CRUD Operations

INSERT Data (Create)

Method

```
public void insertStudent(

    String name,
    int age) {
```



```
SQLiteDatabase db=  
this.getWritableDatabase();  
  
ContentValues cv=  
new ContentValues();  
  
cv.put("name", name);  
  
cv.put("age", age);  
  
db.insert(  
"Student",  
null,  
cv);  
}
```

Explanation

ContentValues

Stores column values.

Example:

```
name = Ali  
  
age = 20
```

Insert Example

```
dbHelper.insertStudent(  
"Ali",  
20);
```

Database After Insert

ID Name Age

1 Ali 20

READ Data

Method

```
public Cursor getStudents() {  
  
    SQLiteDatabase db=  
    this.getReadableDatabase();  
  
    return db.rawQuery(  
    "SELECT * FROM Student",  
    null);  
  
}
```

Cursor

Definition

Cursor is used to read rows returned by a query.

Example

```
Cursor cursor=  
dbHelper.getStudents();
```

Reading Cursor Data

```
while(cursor.moveToNext()) {  
  
    String name=  
  
    cursor.getString(  
    1);  
  
    int age=  
  
    cursor.getInt(  
    2);  
  
}
```

Output

Ali

20

UPDATE Data

Method

```
public void updateStudent(  
  
    int id,  
    String name,  
    int age) {  
  
    SQLiteDatabase db=  
        this.getWritableDatabase();  
  
    ContentValues cv=  
        new ContentValues();  
  
    cv.put("name", name);  
  
    cv.put("age", age);  
  
    db.update(  
        "Student",  
        cv,  
        "id=?",  
        new String[]{  
            String.valueOf(id)  
        });  
}
```

Example

Before:

ID Name Age

1 Ali 20

After:

```
updateStudent(  
    1,  
    "Ali Khan",  
    21);
```

ID	Name	Age
----	------	-----

1	Ali Khan	21
---	----------	----

DELETE Data

Method

```
public void deleteStudent(  
int id) {  
  
    SQLiteDatabase db=  
this.getWritableDatabase();  
  
    db.delete(  
        "Student",  
        "id=?",  
        new String[]{  
String.valueOf(id)  
        });  
}
```

Example

Before:

ID	Name
----	------

1	Ali
---	-----

2	Sara
---	------

Delete:

```
deleteStudent(1);
```

After:

ID	Name
----	------

2	Sara
---	------

Complete CRUD Flow

Insert Student

↓

Database

↓

Read Student

↓

Update Student

↓

Delete Student

Full Practical Example

Student Management App

Features:

Add Student

Name: Ali

Age: 20

[SAVE]

View Students

Ali

Sara

Ahmed

Update Student

Ali

↓

Edit

↓

Save

Delete Student

Long Press

↓

Delete

Using SQLite with RecyclerView

Most Asked Practical Question

Flow:

SQLite Database

↓

Cursor

↓

ArrayList

↓

RecyclerView Adapter

↓

RecyclerView

Real World Applications

WhatsApp

Stores:

Messages

Contacts

Chats

using SQLite.

Instagram

Stores:

Draft Posts

Search History

using SQLite.

FoodPanda

Stores:

Cart Items

Recent Orders

using SQLite.

SQLite Advantages

1. Built-in Database

No installation required.

2. Lightweight

Consumes less memory.

3. Fast

Good for mobile apps.

4. Offline Storage

Works without internet.

5. Free

Included with Android.

SQLite Disadvantages

1. Not Suitable for Huge Systems

Large enterprise systems use:

- MySQL
 - PostgreSQL
 - Oracle
-

2. Single User Oriented

Not ideal for many concurrent users.

SQLite vs MySQL

SQLite	MySQL
Local Database	Server Database
Offline	Online
Lightweight	Heavy

SQLite	MySQL
Mobile Apps	Web Apps
No Server Needed	Requires Server

Viva Questions

What is SQLite?

SQLite is Android's built-in relational database.

What is SQLiteOpenHelper?

A helper class used to create and manage databases.

What are CRUD operations?

Create, Read, Update, Delete.

What is Cursor?

Cursor reads rows returned by a query.

Which method creates tables?

onCreate()

MCQs

SQLite is:

✓ Built-in Android Database

Which class manages SQLite databases?

✓ SQLiteOpenHelper

Which method creates tables?

✓ onCreate()

Which object stores column values?

✓ ContentValues

Which class reads query results?

✓ Cursor

Important Exam Question (15 Marks)

Explain SQLite Database in Android. Discuss SQLiteOpenHelper, database creation, CRUD operations (Insert, Read, Update, Delete), Cursor, and provide a complete Student Management System example with code.

15. JSON + REST API + Retrofit (Complete Detailed Notes)

★ Most Important Topic for Modern Android Development

Nowadays, almost every Android app communicates with a server.

Examples:

- WhatsApp → Fetches messages from server.
- Instagram → Loads posts from server.
- Facebook → Retrieves news feed.
- Weather App → Gets weather data from API.

- FoodPanda → Loads restaurants and orders.

To perform these tasks, Android apps use JSON + REST API + Retrofit.

What is JSON?

Definition

JSON (JavaScript Object Notation) is a lightweight data format used to exchange information between a client and a server.

Simple Words

JSON is a structured text format used to send and receive data over the internet.

Real-World Example

Suppose a server sends user information:

Android app reads this JSON and displays the data.

JSON Structure

1. JSON Object

Uses { }.

Example:

Key-Value Pairs:

Key Value

name Sara

age 22

2. JSON Array

Uses [].

Example:

3. Array of Objects

Most commonly used in APIs.

Client-Server Architecture

Android App (Client)

↓ HTTP Request

Web Server / API

↓ JSON Response

Android App Displays Data

Example

Weather App:

- User opens app.
- App sends request.
- Server returns JSON.
- App displays temperature.

What is REST API?

Definition

REST API is a web service that allows applications to communicate over HTTP using URLs.

Simple Words

REST API is a bridge between your Android app and the server.

Example URL

`https://api.example.com/users`

This endpoint may return all users.

HTTP Methods

1. GET

Purpose: Retrieve data.

Example:

GET /users

Response:

2. POST

Purpose: Send new data.

Request:

Server creates a new user.

3. PUT

Purpose: Update existing data.

Example:

PUT /users/1

4. DELETE

Purpose: Remove data.

Example:

DELETE /users/1

REST API Flow

Android App

↓ GET Request

Server

↓ JSON Response

Retrofit

↓ Java Objects

RecyclerView

↓

User Sees Data

What is Retrofit?

Definition

Retrofit is a type-safe HTTP client library for Android developed by Square.

Why Retrofit?

Without Retrofit:

- Complex networking code.
- Manual JSON parsing.
- More bugs.

With Retrofit:

- Easy API calls.
- Automatic JSON conversion.
- Cleaner code.
- Asynchronous requests.

Add Retrofit Dependency

Module-level build.gradle:

Explanation:

- retrofit: Makes HTTP requests.
- converter-gson: Converts JSON into Java objects.

Step 1: Create Model Class

User.java

This class represents JSON data.

JSON Example

Retrofit automatically maps:

- `id` → `id`
- `name` → `name`
- `email` → `email`

Step 2: Create API Interface

ApiService.java

Explanation:

- `@GET` → HTTP GET request.
- `users` → Endpoint.
- `Call<List<User>>` → Returns list of users.

Step 3: Create Retrofit Instance

RetrofitClient.java

Explanation:

- `baseUrl()` → Server address.
- `addConverterFactory()` → Enables JSON parsing.
- `build()` → Creates Retrofit object.

Step 4: Call API

MainActivity.java

Output

Ali

Ahmed

Sara

How enqueue() Works

Request Sent

↓

Server Processing

↓

Response Received

↓

onResponse()

↓

Data Displayed

If network fails:

Request Sent

↓

Error Occurred

↓

onFailure()

POST Request Example

API Interface

Calling POST

Retrofit converts the object into JSON automatically.

Retrofit Annotations

Annotation	Purpose
@GET	Retrieve data
@POST	Insert data
@PUT	Update data
@DELETE	Delete data
@Body	Send object
@Path	URL parameter
@Query	Query parameter

Example of @Path

Calling:

URL becomes:

/users/5

Example of @Query

Calling:

URL becomes:

/users?name=Ali

Retrofit + RecyclerView

Most Common Practical Question

Flow:

API

↓

Retrofit

↓

List<User>

↓

Adapter

↓

RecyclerView

↓

Screen

Example

- Fetch users from API.
- Store in ArrayList.
- Pass list to Adapter.
- Display in RecyclerView.

Real-World Examples

1. Weather App

Open App



Call Weather API



Receive JSON



Display Temperature

2. News App

Call News API



Get Articles JSON



Show Headlines

3. Instagram

Request Feed



Server Returns JSON



RecyclerView Displays Posts

Advantages of Retrofit

- Simple API calls.
- Automatic JSON conversion.
- Asynchronous networking.
- Less boilerplate code.
- Easy integration with RecyclerView.
- Supports authentication and interceptors.

Disadvantages

- Requires internet connection.
- Learning annotations takes time.
- Debugging network issues can be challenging.

Viva Questions

- What is JSON? → Lightweight data exchange format.
- What is REST API? → Web service that communicates over HTTP.
- What is Retrofit? → Type-safe HTTP client for Android.
- Which converter is commonly used with Retrofit? → Gson Converter.
- Difference between GET and POST? → GET retrieves data, POST sends new data.
- Which callback handles successful responses? → `onResponse()`.
- Which callback handles failures? → `onFailure()`.

MCQs

- JSON stands for: → JavaScript Object Notation ✓
- Which HTTP method retrieves data? → GET ✓
- Which library is used for networking in Android? → Retrofit ✓
- Which annotation sends data to server? → `@POST` + `@Body` ✓
- Which converter parses JSON automatically? → Gson Converter ✓

Important Exam Question (15 Marks)

Explain JSON, REST API, and Retrofit in Android. Discuss HTTP methods, Retrofit architecture, annotations, API calling process, JSON parsing, and RecyclerView integration with suitable examples and code snippets.

16. Notifications in Android (Complete Detailed Notes)

★ Very Important Topic for Theory + Practical Exams

Every modern Android application uses notifications.

Examples:

- WhatsApp Message Notification
- Gmail Email Notification
- Facebook Like Notification
- Daraz Order Notification

- FoodPanda Delivery Notification
-

What is a Notification?

Definition

A Notification is a message displayed outside the application to inform the user about an event.

Simple Words

A notification alerts the user even when the app is closed.

Real World Example

WhatsApp:

Ali: Hello

appears at the top of screen.

This is a notification.

Why Notifications are Used?

Notifications help users:

- Receive updates
 - View alerts
 - Get reminders
 - Track orders
 - Receive messages
-

Notification Architecture

Android App

↓

NotificationManager

↓

Notification

↓

Notification Panel

↓

User

Types of Notifications

1. Basic Notification

Simple title and message.

Example:

Title: New Message

Text: Hello

2. Big Text Notification

Shows large content.

Example:

Gmail email preview.

3. Big Picture Notification

Displays image.

Example:

Instagram photo notification.

4. Action Notification

Contains buttons.

Example:

New Message

[Reply] [Mark Read]

5. Heads-Up Notification

Appears on top of screen.

Example:

Incoming WhatsApp call.

6. Foreground Service Notification

Cannot be removed easily.

Example:

Google Maps Navigation

Spotify Music Playing

Main Classes Used

Class	Purpose
NotificationManager	Shows notifications
NotificationCompat.Builder	Creates notification
NotificationChannel	Android 8+ support
PendingIntent	Opens activity

Class	Purpose
Notification	Final notification object

Notification Flow

Event Occurs

↓

Notification Created

↓

NotificationManager

↓

Notification Panel

↓

User Clicks

↓

Activity Opens

Step 1: Create Notification Channel

★ Required for Android Oreo (API 26+) and above.

Why Channel?

Android groups notifications.

Example:

WhatsApp

Messages

Calls

Groups

Each can have separate channels.

Code

```
if (Build.VERSION.SDK_INT
    >= Build.VERSION_CODES.O) {

    NotificationChannel channel=

    new NotificationChannel(

        "channel1",

        "General Notifications",

        NotificationManager.IMPORTANCE_HIGH);

    NotificationManager manager=

    getSystemService(
        NotificationManager.class);

    manager.createNotificationChannel(
        channel);

}
```

Explanation

"channel1"

Unique Channel ID.

"General Notifications"

Channel Name.

IMPORTANCE_HIGH

High Priority.

Step 2: Create Notification

Basic Notification

```
NotificationCompat.Builder builder=  
  
new NotificationCompat.Builder(  
this,  
"channel1")  
  
.setSmallIcon(  
R.drawable.ic_launcher_foreground)  
  
.setContentTitle(  
"New Message")  
  
.setContentText(  
"Hello from Android")  
  
.setPriority(  
NotificationCompat.PRIORITY_HIGH);
```

Output

```
New Message  
  
Hello from Android
```

Step 3: Show Notification

```
NotificationManagerCompat manager=  
  
NotificationManagerCompat.from(  
this);  
  
manager.notify(  
1,  
builder.build());
```

Explanation

1

Notification ID.

```
builder.build()
```

Creates notification object.

Complete Example

```
NotificationCompat.Builder builder=

new NotificationCompat.Builder(
this,
"channel1")

.setSmallIcon(
R.drawable.ic_launcher_foreground)

.setContentTitle(
"Welcome")

.setContentText(
"Android Notification")

.setPriority(
NotificationCompat.PRIORITY_HIGH);

NotificationManagerCompat manager=

NotificationManagerCompat.from(
this);

manager.notify(
1,
builder.build());
```

Output

Welcome

Android Notification

PendingIntent

Definition

PendingIntent allows another application or Android system to execute your app's action later.

Simple Words

When user clicks notification:

Open Activity.

This is done using PendingIntent.

Example

```
Intent intent=
new Intent(
this,
MainActivity.class);

PendingIntent pendingIntent=

PendingIntent.getActivity(
this,
0,
intent,
PendingIntent.FLAG_IMMUTABLE);
```

Attach PendingIntent

```
builder.setContentIntent(
pendingIntent);
```

Flow

Notification Click

↓

PendingIntent

↓

MainActivity Opens

Action Buttons

Example

WhatsApp:

New Message

[Reply]

[Mark Read]

Code

```
builder.addAction(  
    R.drawable.ic_reply,  
    "Reply",  
    pendingIntent);
```

Big Text Notification

Useful for long messages.

Example

```
builder.setStyle(  
    new NotificationCompat.BigTextStyle()  
        .bigText(  
            "This is a very long message shown inside notification."));
```

Output

Large expanded text.

Big Picture Notification

Useful for images.

Example

```
Bitmap bitmap=

BitmapFactory.decodeResource(
    getResources(),
    R.drawable.photo);

builder.setStyle(

    new NotificationCompat.BigPictureStyle()

        .bigPicture(bitmap));
```

Real World Example

Instagram:

New Post

[Image Preview]

Heads-Up Notification

Appears immediately on screen.

Requirements

IMPORTANCE_HIGH

and

PRIORITY_HIGH

Real World Example

Incoming Call:

Ali Calling...

[Accept] [Reject]

Foreground Service Notification

Required for background services.

Example

Spotify

Now Playing

Song Name

Google Maps

Navigation Active

Cancel Notification

Example

```
NotificationManagerCompat manager=
```

```
NotificationManagerCompat.from(  
this);
```

```
manager.cancel(1);
```

Output

Notification disappears.

Update Notification

Use same notification ID.

```
manager.notify(  
1,  
updatedBuilder.build());
```

Old notification gets updated.

Real World Examples

WhatsApp

Ali: Hi

↓

Notification

Gmail

New Email

↓

Notification

FoodPanda

Order Delivered

↓

Notification

Daraz

Flash Sale

↓

Notification

Firestore Cloud Messaging (FCM)

Definition

FCM allows server-to-device notifications.

Flow

Server

↓

Firestore Cloud Messaging

↓

Android Device

↓

Notification

Example

Instagram Server

↓

FCM

↓

Phone

↓

"You received a new follower"

Local vs Push Notifications

Local Notification	Push Notification
Generated by App	Generated by Server
Offline	Requires Internet
Alarm Reminder	WhatsApp Message
Calendar Alert	Instagram Notification

Notification Lifecycle

```text  
Create Notification

↓

Display Notification

↓

User Views

↓

Click Notification

↓

Activity Opens

↓

Notification Removed

---

# Advantages

## 1. User Engagement

Keeps users active.

---

## **2. Instant Updates**

Messages arrive instantly.

---

## **3. Better Experience**

Important information delivered quickly.

---

## **4. Marketing**

Promotions and offers.

---

# **Disadvantages**

## **1. Too Many Notifications**

Users may become annoyed.

---

## **2. Battery Usage**

Frequent push notifications consume battery.

---

## **3. Privacy Issues**

Sensitive information may appear on lock screen.

---

# **Viva Questions**

## **What is a Notification?**

A message displayed outside the application.

---

---

**Which class displays notifications?**

NotificationManager

---

**What is NotificationChannel?**

Used to categorize notifications on Android 8+.

---

**What is PendingIntent?**

An intent executed later by Android.

---

**Which service sends push notifications?**

Firebase Cloud Messaging (FCM)

---

## MCQs

**Which class manages notifications?**

✓ NotificationManager

---

**Which class builds notifications?**

✓ NotificationCompat.Builder

---

**Which component opens Activity from notification?**

✓ PendingIntent

---

---

**Which Android version introduced NotificationChannel?**

✓ Android Oreo (API 26)

---

**Which service provides push notifications?**

✓ Firebase Cloud Messaging (FCM)

---

## Important Exam Question (15 Marks)

**Explain Notifications in Android. Discuss NotificationManager, NotificationChannel, PendingIntent, Action Buttons, Big Text Notifications, Big Picture Notifications, Foreground Services, and Firebase Cloud Messaging with suitable examples and code snippets.**

## 7. Fragments in Android (Complete Detailed Notes)

★ **Very Important Topic for Theory + Practical Exams**

Modern Android apps rarely use only Activities. Instead, they use **Fragments**.

Examples:

- YouTube
- Gmail
- WhatsApp
- Instagram
- Facebook

All heavily use Fragments.

---

## What is a Fragment?

## Definition

A Fragment is a reusable portion of a user interface that exists inside an Activity.

---

## Simple Words

A Fragment is like a mini Activity that lives inside an Activity.

---

## Real World Example

### YouTube App

Bottom Navigation:

Home | Shorts | Subscriptions | Library

When you click:

Home

Home Fragment loads.

When you click:

Library

Library Fragment loads.

Same Activity remains open.

Only Fragment changes.

---

## Why Fragments are Used?

Without Fragments:

Home Activity

↓

---

Open Another Activity

↓

Open Another Activity

Many Activities.

More memory usage.

---

With Fragments:

MainActivity

↓

Home Fragment

↓

Profile Fragment

↓

Settings Fragment

Better performance.

---

## Activity vs Fragment

| Activity           | Fragment           |
|--------------------|--------------------|
| Independent screen | Part of screen     |
| Can exist alone    | Needs Activity     |
| Heavy              | Lightweight        |
| Own lifecycle      | Fragment lifecycle |
| More memory        | Less memory        |

---

## Fragment Architecture

MainActivity

```
|
|— HomeFragment
|— ProfileFragment
|— SettingsFragment
```

---

## Real World Example

Instagram

MainActivity

```
|
|— Home Fragment
|— Search Fragment
|— Reels Fragment
|— Profile Fragment
```

Only fragments switch.

Activity remains same.

---

## Fragment Lifecycle

Very Important Exam Question

---

### Lifecycle Flow

```
onAttach()
↓
onCreate()
↓
onCreateView()
```

↓  
onViewCreated()  
↓  
onStart()  
↓  
onResume()  
(App Running)  
↓  
onPause()  
↓  
onStop()  
↓  
onDestroyView()  
↓  
onDestroy()  
↓  
onDetach()

---

## 1. onAttach()

Fragment attached to Activity.

```
@Override
public void onAttach(Context context) {
 super.onAttach(context);
}
```

---

## 2. onCreate()

Initialize variables.

```
@Override
public void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
}
```

---



```
}
```

---

## 3. onCreateView() ★

Most Important Method

Used to load UI.

```
@Override

public View onCreateView(

 LayoutInflater inflater,

 ViewGroup container,

 Bundle savedInstanceState) {

 return inflater.inflate(
 R.layout.fragment_home,
 container,
 false);

}
```

---

## What Happens?

fragment\_home.xml

↓

Inflated

↓

Displayed

---

## Example

```
<TextView
 android:text="Home Fragment"/>
```

Output:

Home Fragment

---

---

# Creating a Fragment

Android Studio

New

↓

Fragment

↓

Fragment (Blank)

Creates:

HomeFragment.java

fragment\_home.xml

---

## Fragment Structure

HomeFragment.java

fragment\_home.xml

---

## Example Fragment

### HomeFragment.java

```
public class HomeFragment
extends Fragment {

 @Override

 public View onCreateView(

 LayoutInflater inflater,

 ViewGroup container,

 Bundle savedInstanceState) {

 return inflater.inflate(
```

```
R.layout.fragment_home,
container,
false);

}

}
```

---

## fragment\_home.xml

```
<LinearLayout

 android:layout_width="match_parent"

 android:layout_height="match_parent">

 <TextView

 android:text="Home Fragment"

 android:textSize="24sp"/>

</LinearLayout>
```

---

## Output

Home Fragment

---

## Adding Fragment to Activity

### activity\_main.xml

```
<FrameLayout
 android:id="@+id/frameLayout"
 android:layout_width="match_parent"
 android:layout_height="match_parent"/>
```

---

## Why FrameLayout?

Fragment needs a container.

FrameLayout acts as container.

---

---

# Add Fragment Programmatically

## MainActivity.java

```
getSupportFragmentManager()

.beginTransaction()

.add(
 R.id.frameLayout,
 new HomeFragment()

.commit();
```

---

## Output

FrameLayout

↓

HomeFragment

---

# FragmentManager

## Definition

FragmentManager manages fragments.

Used for:

- Add Fragment
  - Remove Fragment
  - Replace Fragment
- 

## Example

```
FragmentManager manager=
```

---

```
getSupportFragmentManager();
```

---

# FragmentTransaction

## Definition

Used to perform fragment operations.

---

## Example

```
FragmentTransaction transaction=
getSupportFragmentManager()
.beginTransaction();
```

---

## Operations

```
add()
replace()
remove()
```

---

## Add Fragment

```
transaction.add(
R.id.frameLayout,
new HomeFragment());
transaction.commit();
```

---

## Replace Fragment ★

Most Asked Practical Question

---

## Example

```
transaction.replace(
 R.id.frameLayout,
 new ProfileFragment());

transaction.commit();
```

---

## Flow

Home Fragment

↓

Replace

↓

Profile Fragment

---

## Remove Fragment

```
transaction.remove(
 fragment);

transaction.commit();
```

---

## Fragment Navigation Example

Button Click

↓

Home Fragment

↓

Profile Fragment

---

## Java Code

```
btnProfile.setOnClickListener(v -> {

 getSupportFragmentManager()

 .beginTransaction()

 .replace(
 R.id.frameLayout,
 new ProfileFragment()
)

 .commit();

});
```

---

## Passing Data Activity → Fragment

Very Important

---

### Activity

```
Bundle bundle=
 new Bundle();

bundle.putString(
 "name",
 "Ali");
```

---

### Send Bundle

```
HomeFragment fragment=
 new HomeFragment();

fragment.setArguments(bundle);
```

---

### Fragment Receive

```
String name=

 getArguments()

 .getString("name");
```

---

# Output

Ali

---

## Passing Data Fragment → Activity

### Interface Method

```
public interface OnDataPass {

 void onDataPass(
 String data);

}
```

---

Activity implements interface.

Fragment sends data.

---

## Fragment Back Stack

### Problem

User navigates:

Home

↓

Profile

↓

Settings

Press Back.

App exits.

---



## Solution

```
.addToBackStack(null)
```

---

## Example

```
getSupportFragmentManager()

.beginTransaction()

.replace(
R.id.frameLayout,
new ProfileFragment())

.addToBackStack(null)

.commit();
```

---

## Flow

Home

↓

Profile

↓

Back

↓

Home

---

## Fragment with Bottom Navigation

Most Common Real App Example

---

## YouTube

Home

Shorts

---

Subscriptions

Library

Each tab = Fragment.

---

## Flow

Bottom Navigation

↓

Selected Item

↓

Replace Fragment

↓

Display Fragment

---

## Fragment with Navigation Drawer

Example:

Home

Profile

Settings

Logout

Each menu item loads a fragment.

---

## Real World Examples

### YouTube

Home Fragment

---

Shorts Fragment

Library Fragment

---

## Instagram

Home Fragment

Search Fragment

Reels Fragment

Profile Fragment

---

## Gmail

Inbox Fragment

Sent Fragment

Starred Fragment

---

# Advantages of Fragments

## Reusable

Same fragment used multiple times.

---

## Better Performance

Less memory than Activities.

---

## Tablet Support

Multiple fragments on one screen.

---

## Dynamic UI

---

Easy screen updates.

---

## Disadvantages

### Lifecycle Complexity

More methods than Activities.

---

### Communication Complexity

Passing data requires extra work.

---

## Fragment Lifecycle vs Activity Lifecycle

Activity	Fragment
onCreate()	onCreate()
onStart()	onStart()
onResume()	onResume()
onPause()	onPause()
onStop()	onStop()
onDestroy()	onDestroy()
—	onCreateView()
—	onDestroyView()

---

## Viva Questions

### What is a Fragment?

A reusable UI component that exists inside an Activity.

---

### Can Fragment run independently?

No.

It needs an Activity.

---

**Which method loads Fragment UI?**

`onCreateView()`

---

**Which class manages Fragments?**

`FragmentManager`

---

**Which method replaces a Fragment?**

`replace()`

---

**Which method adds Fragment to Back Stack?**

`addToBackStack()`

---

## MCQs

**Fragment is:**

✓ Part of Activity

---

**Which method creates Fragment UI?**

✓ `onCreateView()`

---

**Which class manages Fragments?**

✓ `FragmentManager`

---

**Which method replaces Fragment?**

✓ replace()

---

**Which layout commonly hosts Fragments?**

✓ FrameLayout

---

## **Important Exam Question (15 Marks)**

**Explain Fragments in Android. Discuss Fragment Lifecycle, FragmentManager, FragmentTransaction, Add/Replace Fragments, Fragment Communication, Back Stack, and Bottom Navigation with suitable examples and code snippets.**

## **18. Menus in Android (Complete Detailed Notes)**

★ **Very Important Topic for Theory + Practical Exams**

Menus are used in almost every Android application.

Examples:

- WhatsApp (Settings, New Group, Linked Devices)
  - Gmail (Refresh, Settings)
  - YouTube (Share, Report, Save)
  - Facebook (Profile, Logout)
- 

## **What is a Menu?**

### **Definition**

A Menu is a collection of options or commands presented to the user.

---

## Simple Words

A menu provides different actions that users can perform inside an application.

---

## Real World Example

### WhatsApp Menu

:

New Group  
New Broadcast  
Linked Devices  
Settings

This list of options is called a Menu.

---

## Why Menus are Used?

Menus help to:

- Save screen space
  - Organize features
  - Improve user experience
  - Provide quick access to actions
- 

## Types of Menus in Android

Android provides three main menu types:

Menus

└ Options Menu  
└ Context Menu  
└ Popup Menu

---

## 1. Options Menu

## Definition

The Options Menu appears in the Action Bar or Toolbar.

Usually shown using:

:

(Three dots menu)

---

## Real World Example

### WhatsApp

:

New Group  
Linked Devices  
Settings

---

## Creating Options Menu

---

### Step 1: Create Menu Resource

Create:

```
res
├── menu
│ └── main_menu.xml
```

---

### main\_menu.xml

```
<menu xmlns:android=
"http://schemas.android.com/apk/res/android">

<item
android:id="@+id/menu_settings"
android:title="Settings"/>

<item
```



```
android:id="@+id/menu_about"
android:title="About"/>

</menu>
```

---

## Output

:

Settings  
About

---

## Step 2: Inflate Menu

Inside MainActivity:

```
@Override

public boolean onCreateOptionsMenu(
 Menu menu) {

 getMenuInflater().inflate(
 R.menu.main_menu,
 menu);

 return true;
}
```

---

## What Happens?

main\_menu.xml

↓

Inflated

↓

Displayed on Toolbar

---

## Step 3: Handle Menu Click

```
@Override
```

---

```
public boolean onOptionsItemSelected(
MenuItem item) {

 if(item.getItemId()
 == R.id.menu_settings){

 Toast.makeText(
 this,
 "Settings Clicked",
 Toast.LENGTH_SHORT).show();

 }

 return true;

}
```

---

## Output

Settings Clicked

---

## Full Flow

User Clicks :

↓

Options Menu Opens

↓

Select Settings

↓

onOptionsItemSelected()

↓

Action Performed

---

## 2. Context Menu

### Definition

Context Menu appears when user long-presses a View.

---

## Real World Example

### WhatsApp Message

Long Press Message



Copy

Delete

Forward

This is Context Menu.

---

## Creating Context Menu

---

### XML

```
<TextView
 android:id="@+id/txtMessage"
 android:text="Hello"/>
```

---

## Register View

```
TextView txt;

txt=findViewById(
 R.id.txtMessage);

registerForContextMenu(txt);
```

---

## Create Context Menu

```
@Override
```

---

```
public void onCreateContextMenu(

 ContextMenu menu,

 View v,

 ContextMenu.ContextMenuInfo menuInfo) {

 super.onCreateContextMenu(
 menu,
 v,
 menuInfo);

 menu.add(
 0,
 1,
 0,
 "Copy");

 menu.add(
 0,
 2,
 0,
 "Delete");

}
```

---

## Output

Long Press:

Copy

Delete

---

## Handle Context Menu Click

```
@Override

public boolean onContextItemSelected(
 MenuItem item) {

 if (item.getItemId() == 1) {

 Toast.makeText(
 this,
 "Copied",
 Toast.LENGTH_SHORT).show();

 }

}
```

```
return true;
}
```

---

## Output

Copied

---

## Context Menu Flow

Long Press

↓

Context Menu

↓

Select Item

↓

Action

---

## 3. Popup Menu

### Definition

Popup Menu appears attached to a specific view.

Usually displayed after clicking a button.

---

### Real World Example

Instagram Post

:

Save

Report

---

Share

Popup Menu appears near the clicked item.

---

## Create Popup Menu

### XML

```
<Button
android:id="@+id/btnMenu"
android:text="Menu"/>
```

---

### Java Code

```
Button btnMenu;

btnMenu=findViewById(
R.id.btnMenu);

btnMenu.setOnClickListener(v -> {

 PopupMenu popup=

 new PopupMenu(
 this,
 btnMenu);

 popup.getMenu().add("Edit");

 popup.getMenu().add("Delete");

 popup.show();

});
```

---

### Output

Edit

Delete

---

# Handle Popup Click

```
popup.setOnMenuItemClickListener(

 item -> {

 Toast.makeText(
 this,
 item.getTitle(),
 Toast.LENGTH_SHORT).show();

 return true;

 });
```

---

## Output

Edit

Clicked.

---

## Menu XML Attributes

### Example

```
<item
 android:id="@+id/menu_profile"
 android:title="Profile"
 android:icon="@drawable/ic_user"/>
```

---

## Important Attributes

Attribute	Purpose
id	Unique identifier
title	Menu text
icon	Menu icon
visible	Show/hide item
enabled	Enable/disable item

---

# Toolbar Menu

Modern Android uses Toolbar instead of old ActionBar.

---

## XML

```
<android.support.design.widget.Toolbar
 android:id="@+id/toolbar"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"/>
```

---

## Set Toolbar

```
Toolbar toolbar=
 findViewById(R.id.toolbar);

setSupportActionBar(toolbar);
```

---

## Output

Custom toolbar with menu.

---

## Action Bar vs Toolbar

Action Bar	Toolbar
Default	Customizable
Less Flexible	Highly Flexible
Old Approach	Modern Approach

---

## Menu with Icons

```
<item
```

---



```
android:id="@+id/menu_home"
android:title="Home"
android:icon="@drawable/ic_home"/>
```

---

## Real World Example

Gmail

🔍 Search

⚙️ Settings

---

## Dynamic Menu

Menu items can be added at runtime.

---

### Example

```
menu.add(
0,
1,
0,
"Logout");
```

---

## Output

Logout

appears dynamically.

---

## Real World Applications

### WhatsApp

New Group

Settings

---

Linked Devices

Options Menu.

---

## Instagram

Save

Report

Share

Popup Menu.

---

## Gmail

Delete

Archive

Mark Read

Context Menu.

---

# Advantages of Menus

## 1. Organized Interface

Features are grouped.

---

## 2. Saves Space

No need for many buttons.

---

## 3. Easy Navigation

---

Users quickly access functions.

---

#### **4. Better User Experience**

Cleaner UI.

---

## **Disadvantages**

### **1. Hidden Features**

Users may not notice menu items.

---

### **2. Too Many Options**

Can become confusing.

---

## **Viva Questions**

### **What is a Menu?**

A collection of actions available to the user.

---

### **How many main menu types exist in Android?**

Three:

- Options Menu
  - Context Menu
  - Popup Menu
- 

### **Which menu appears on long press?**

Context Menu.

---

**Which menu appears near a view?**

Popup Menu.

---

**Which method inflates Options Menu?**

`onCreateOptionsMenu()`

---

**Which method handles menu clicks?**

`onOptionsItemSelected()`

---

## MCQs

**Which menu appears in Toolbar?**

✓ Options Menu

---

**Which menu appears on long press?**

✓ Context Menu

---

**Which class creates popup menus?**

✓ PopupMenu

---

**Which method inflates menu XML?**

✓ `onCreateOptionsMenu()`

---

---

**Which method handles menu selection?**

✓ onOptionsItemSelected()

---

## **Important Exam Question (15 Marks)**

**Explain Menus in Android. Discuss Options Menu, Context Menu, Popup Menu, Toolbar Menus, Menu XML, menu event handling, advantages, and provide complete code examples.**

## **9. Dialogs in Android (Complete Detailed Notes)**

★ **Very Important Topic for Theory + Practical Exams**

Dialogs are used in almost every Android application to interact with users.

Examples:

- Delete Confirmation
- Logout Confirmation
- Date Selection
- Time Selection
- Loading Screen
- Custom Login Popup

---

## **What is a Dialog?**

### **Definition**

A Dialog is a small popup window that appears on top of an Activity to get user input or display important information.

---

## Simple Words

A Dialog is a popup message box that asks the user to make a decision.

---

## Real World Example

### WhatsApp Delete Message

Delete Message?

[YES]      [NO]

This popup is called a Dialog.

---

## Why Dialogs are Used?

Dialogs help:

- Show important messages
  - Take user confirmation
  - Select date and time
  - Show loading progress
  - Collect user input
- 

## Types of Dialogs in Android

Dialogs

- AlertDialog
- Custom Dialog
- ProgressDialog
- DatePickerDialog
- TimePickerDialog
- DialogFragment

---

## 1. AlertDialog ★ MOST IMPORTANT

## Definition

AlertDialog is used to display messages and get user confirmation.

---

## Real World Example

Instagram Logout

Are you sure you want to logout?

[YES]

[NO]

---

## AlertDialog Structure

-----  
Title

Message

[Positive Button]

[Negative Button]  
-----

---

## Example

### Java Code

```
AlertDialog.Builder builder=
new AlertDialog.Builder(this);

builder.setTitle(
"Delete");

builder.setMessage(
"Are you sure?");

builder.setPositiveButton(
"Yes",
```

```
(dialog, which) -> {

 Toast.makeText(
 this,
 "Deleted",
 Toast.LENGTH_SHORT).show();

 });

 builder.setNegativeButton(
 "No",

 (dialog, which) -> {

 dialog.dismiss();

 });

 builder.show();
}
```

---

## Output

```
Delete
Are you sure?
[YES]
[NO]
```

---

## AlertDialog Buttons

### Positive Button

Used for:

```
YES
OK
SAVE
```

---

### Negative Button

Used for:



NO

CANCEL

---

## Neutral Button

Used for:

LATER

REMIND ME

---

## Example with Neutral Button

```
builder.setNeutralButton(
 "Later",
 (dialog, which) -> {
 Toast.makeText(
 this,
 "Later Selected",
 Toast.LENGTH_SHORT).show();
 });
```

---

## Output

Delete

Are you sure?

YES

NO

LATER

---

## AlertDialog Flow

User Clicks Delete

↓

---

AlertDialog Opens

↓

User Selects Yes

↓

Delete Action Performed

---

## 2. Custom Dialog ★ PRACTICAL IMPORTANT

### Definition

Custom Dialog uses a custom XML layout.

---

### Why Use Custom Dialog?

Default dialog is limited.

Custom dialog allows:

- TextBoxes
  - Buttons
  - Images
  - Forms
- 

### Real World Example

Login Popup

Username

Password

[LOGIN]

---

# Step 1: Create Layout

## dialog\_login.xml

```
<LinearLayout
 android:orientation="vertical"
 android:padding="20dp"
 android:layout_width="match_parent"
 android:layout_height="wrap_content">
 <EditText
 android:id="@+id/etUser"
 android:hint="Username"/>
 <EditText
 android:id="@+id/etPass"
 android:hint="Password"/>
 <Button
 android:id="@+id/btnLogin"
 android:text="Login"/>
</LinearLayout>
```

---

# Step 2: Show Custom Dialog

```
Dialog dialog=
 new Dialog(this);

dialog.setContentView(
 R.layout.dialog_login);

dialog.show();
```

---

# Output

Username

Password

[LOGIN]

---

## Access Dialog Views

```
EditText user=

dialog.findViewById(
 R.id.etUser);

Button btn=

dialog.findViewById(
 R.id.btnLogin);
```

---

## Login Button Click

```
btn.setOnClickListener(v -> {

 String name=

 user.getText().toString();

 Toast.makeText(
 this,
 name,
 Toast.LENGTH_SHORT).show();

});
```

---

## Output

Input:

Ali

Output:

Ali

---

## 3. ProgressDialog

## Definition

Shows loading progress.

---

## Real World Example

Downloading File

Loading...

Please Wait

---

## Example

```
ProgressDialog progress=
new ProgressDialog(this);

progress.setTitle(
"Loading");

progress.setMessage(
"Please Wait");

progress.show();
```

---

## Output

Loading

Please Wait

---

## Close Progress Dialog

```
progress.dismiss();
```

---

## Flow

Start Download

---

↓

Show ProgressDialog

↓

Download Complete

↓

Dismiss Dialog

---

## Note

ProgressDialog is deprecated.

Nowadays:

ProgressBar

is preferred.

---

## 4. DatePickerDialog ★ VERY IMPORTANT

### Definition

Used to select a date.

---

### Real World Example

FoodPanda Delivery Date

Birthday Selection

Hotel Booking

---

# Example

```
DatePickerDialog picker=
new DatePickerDialog(
this,

(datePicker,
year,
month,
day) ->{

 Toast.makeText(
this,

day+"/"+month+"/"+year,

 Toast.LENGTH_SHORT).show();

},

2025,
7,
1);

picker.show();
```

---

# Output

Select Date  
01/08/2025

---

# Flow

Open DatePicker  
↓  
Choose Date  
↓  
Date Returned

---

## 5. TimePickerDialog

### Definition

Used to select time.

---

### Real World Example

Alarm Application

Meeting Scheduler

---

### Example

```
TimePickerDialog picker=
new TimePickerDialog(
 this,
 (view, hour, minute) -> {
 Toast.makeText(
 this,
 hour + ":" + minute,
 Toast.LENGTH_SHORT).show();
 },
 10,
 30,
 true);
picker.show();
```

---

### Output

10:30

---



---

# Flow

Open TimePicker

↓

Select Time

↓

Time Returned

---

# DialogFragment

## Definition

A Fragment that behaves like a Dialog.

---

## Why DialogFragment?

Traditional Dialog:

Screen Rotation

↓

Dialog Lost

---

DialogFragment:

Screen Rotation

↓

Dialog Preserved

---

## Example

```
public class MyDialog
```

---

```
extends DialogFragment {

 @Override

 public Dialog onCreateDialog(
 Bundle savedInstanceState) {

 return new AlertDialog.Builder(
 getActivity())
 .setTitle("Hello")
 .setMessage("DialogFragment")
 .create();
 }
}
```

---

## Show DialogFragment

```
MyDialog dialog=
 new MyDialog();

dialog.show(
 getSupportFragmentManager(),
 "MyDialog");
```

---

## Real World Examples

### WhatsApp

Delete Chat?

YES

NO

### AlertDialog

---

## Instagram

Logout?

YES

NO

AlertDialog

---

## FoodPanda

Select Delivery Date

DatePickerDialog

---

## Alarm App

Select Time

TimePickerDialog

---

## Daraz

Loading Products...

Progress Dialog

---

## Dialog Lifecycle

Create Dialog

↓

Show Dialog

↓

---

User Interaction

↓

Dismiss Dialog

↓

Destroy Dialog

---

## Advantages of Dialogs

### 1. User Interaction

Easy communication with user.

---

### 2. Confirmation

Prevents accidental actions.

---

### 3. Better UX

Improves user experience.

---

### 4. Compact

Uses less screen space.

---

## Disadvantages

### 1. Interrupts Workflow

Can annoy users.

---

## 2. Too Many Dialogs

Bad user experience.

---

## 3. Complex Custom Dialogs

Require additional coding.

---

# AlertDialog vs Custom Dialog

<b>AlertDialog</b>	<b>Custom Dialog</b>
Built-in	User Defined
Simple	Flexible
Less Customization	Full Customization
Easy	Slightly Difficult

---

# Viva Questions

## What is a Dialog?

A popup window used to interact with users.

---

## Which Dialog is used for confirmation?

AlertDialog

---

## Which Dialog is used for selecting dates?

DatePickerDialog

---

## Which Dialog is used for selecting time?

TimePickerDialog

---

**Which Dialog uses custom XML?**

Custom Dialog

---

**Which Dialog survives configuration changes?**

DialogFragment

---

## MCQs

**Which Dialog is used for confirmation messages?**

✓ AlertDialog

---

**Which Dialog selects dates?**

✓ DatePickerDialog

---

**Which Dialog selects time?**

✓ TimePickerDialog

---

**Which Dialog uses custom layout?**

✓ Custom Dialog

---

**Which class is recommended instead of ProgressDialog?**

---

## Important Exam Question (15 Marks)

**Explain Dialogs in Android. Discuss AlertDialog, Custom Dialog, ProgressDialog, DatePickerDialog, TimePickerDialog, DialogFragment, advantages, disadvantages, and provide complete code examples.**

## 20. Multimedia in Android (Complete Detailed Notes)

★ **Very Important Topic for Theory + Practical Exams**

Modern Android applications heavily use multimedia.

Examples:

- YouTube → Video Playback
  - Spotify → Audio Playback
  - Instagram → Images & Reels
  - WhatsApp → Voice Notes & Images
  - TikTok → Video Streaming
- 

## What is Multimedia?

### Definition

Multimedia refers to the use of multiple forms of media such as:

- Text
- Images
- Audio
- Video
- Animation

inside an Android application.

---

---

## Simple Words

Multimedia means displaying or handling audio, video, images, and animations in Android apps.

---

# Multimedia Components in Android

Multimedia

- Images
  - Audio
  - Video
  - Camera
  - Audio Recording
  - Animation
- 

## Android Multimedia Architecture

User

↓

Android App

↓

Media Framework

↓

Audio / Video / Images

↓

Device Hardware

---

## 1. Image Handling in Android

Images are the most commonly used multimedia content.

Examples:



- Instagram Photos
  - WhatsApp Profile Pictures
  - Facebook Posts
- 

# Displaying an Image

## Step 1: Place Image

```
res
└─ drawable
 └─ photo.png
```

---

## Step 2: XML

```
<ImageView
 android:id="@+id/imgPhoto"
 android:layout_width="200dp"
 android:layout_height="200dp"
 android:src="@drawable/photo"/>
```

---

## Output

[PHOTO DISPLAYED]

---

# ImageView

## Definition

ImageView is used to display images.

---

## Common Attributes

`android:src`

Sets image.

---

---

```
android:scaleType="centerCrop"
```

Adjusts image size.

---

## Example

```
<ImageView
 android:layout_width="200dp"
 android:layout_height="200dp"
 android:src="@drawable/photo"
 android:scaleType="centerCrop"/>
```

---

## Real World Example

Instagram Feed

ImageView

↓

User Photo Display

---

## 2. Audio Playback Using MediaPlayer

★ Most Important Practical Topic

---

## What is MediaPlayer?

MediaPlayer class is used to play audio and video files.

---

## Real World Example

Spotify

Play Song

↓

MediaPlayer

↓

Audio Output

---

## Audio File Location

res

└─ raw

└─ song.mp3

---

## Create raw Folder

res

↓

New

↓

Android Resource Directory

↓

raw

---

## Play Audio

### Java Code

```
MediaPlayer player;

player = MediaPlayer.create(
 this,
 R.raw.song);
```

```
player.start();
```

---

## Output

Song Starts Playing

---

## Pause Audio

```
player.pause();
```

---

## Stop Audio

```
player.stop();
```

---

## Release Resources

```
player.release();
```

---

## Complete Example

### XML

```
<Button
 android:id="@+id/btnPlay"
 android:text="Play"/>

<Button
 android:id="@+id/btnPause"
 android:text="Pause"/>
```

---

### Java

```
MediaPlayer player;

player =
MediaPlayer.create(
 this,
```

```
R.raw.song);

btnPlay.setOnClickListener(v -> {
 player.start();
});

btnPause.setOnClickListener(v -> {
 player.pause();
});
```

---

## Output

Play → Song Plays

Pause → Song Pauses

---

## MediaPlayer Lifecycle

Create

↓

Prepare

↓

Start

↓

Pause

↓

Stop

↓

Release

---

## 3. Video Playback Using VideoView

★ Exam + Practical Important

---

# What is VideoView?

VideoView is used to play videos inside Android applications.

---

## Real World Example

YouTube

Play Video

↓

VideoView

↓

Display Video

---

## Add Video File

res

└─ raw

└─ movie.mp4

---

## XML

```
<VideoView
 android:id="@+id/videoView"
 android:layout_width="match_parent"
 android:layout_height="300dp"/>
```

---

## Java Code

```
VideoView videoView;

videoView =
findViewById(
R.id.videoView);

String path =
"android.resource://" +
getPackageName() +
 "/" +
R.raw.movie;

Uri uri = Uri.parse(path);
videoView.setVideoURI(uri);
videoView.start();
```

---

## Output

Video Starts Playing

---

## Add Media Controls

```
MediaController controller =
new MediaController(this);
videoView.setMediaController(
controller);
controller.setAnchorView(
videoView);
```

---

## Output

Play

Pause

Seek Bar

---

## 4. Audio Recording

Used to record sound from microphone.

---

### Real World Example

WhatsApp Voice Notes

Record

↓

Save Audio

↓

Send

---

## Required Permission

### AndroidManifest.xml

```
<uses-permission
android:name=
"android.permission.RECORD_AUDIO"/>
```

---

## MediaRecorder

Used for recording audio.

---

### Example

```
MediaRecorder recorder =
new MediaRecorder();
```

---

## Recording Flow



Create Recorder

↓

Configure Source

↓

Start Recording

↓

Stop Recording

↓

Save File

---

## 5. Camera Basics

Android allows apps to access device camera.

---

### Real World Examples

- Instagram Camera
  - Snapchat
  - WhatsApp Camera
- 

## Camera Permission

```
<uses-permission
android:name=
"android.permission.CAMERA"/>
```

---

## Open Camera

```
Intent intent =

new Intent(
MediaStore.ACTION_IMAGE_CAPTURE);
```

---

```
startActivityForResult(
intent,
1);
```

---

## Output

Camera Opens

---

## Camera Flow

User Clicks Camera

↓

Camera App Opens

↓

Take Picture

↓

Return Image

---

## 6. Runtime Permissions

★ Android 6.0+ Important

---

## Why Runtime Permissions?

For security reasons.

Users must allow access manually.

---

## Example Permissions

Camera

---

Microphone

Storage

Location

---

## Request Permission

```
ActivityCompat.requestPermissions(
 this,
 new String[]{
 Manifest.permission.CAMERA
 },
 1);
```

---

## Check Permission

```
if (ContextCompat.checkSelfPermission(
 this,
 Manifest.permission.CAMERA)
 == PackageManager.PERMISSION_GRANTED) {
}
```

---

## Permission Flow

App Requests Permission

↓

User Allows

↓

Feature Works

---

# SurfaceView

## Definition

SurfaceView provides a dedicated drawing surface.

Used in:

- Games
- Camera Preview
- Video Streaming

---

## Real World Example

PUBG Mobile

Game Graphics

↓

SurfaceView

---

## Multimedia Applications

### YouTube

Uses:

VideoView

MediaPlayer

---

### Spotify

Uses:

MediaPlayer

---

## WhatsApp

Uses:

Images

Voice Notes

Camera

---

## Instagram

Uses:

Camera

Images

Videos

---

# Advantages of Multimedia

## Rich User Experience

More engaging apps.

---

## Better Communication

Audio and video explain information clearly.

---

## Interactive Applications

Improves usability.

---

## Entertainment

Supports media-rich apps.

---

---

# Disadvantages

## Large Storage Usage

Media files consume memory.

---

## Battery Consumption

Audio/video processing uses power.

---

## Permission Management

Requires camera and microphone permissions.

---

## Performance Issues

High-quality media requires more resources.

---

# ImageView vs VideoView

<b>ImageView</b>	<b>VideoView</b>
Displays images	Displays videos
Lightweight	Heavier
Static content	Dynamic content

---

# MediaPlayer vs VideoView

<b>MediaPlayer</b>	<b>VideoView</b>
Audio + Video Backend	Video UI Component
No UI	Has Video Display

---

<b>MediaPlayer</b>	<b>VideoView</b>
Used internally	User-facing

---

## Viva Questions

### What is Multimedia?

Combination of text, audio, video, images, and animation.

---

### Which class plays audio?

MediaPlayer

---

### Which view displays videos?

VideoView

---

### Which class records audio?

MediaRecorder

---

### Which permission is required for camera?

android.permission.CAMERA

---

### Which permission records audio?

android.permission.RECORD\_AUDIO

---

### What is SurfaceView?

A view used for drawing graphics and displaying media efficiently.

---

## MCQs

**Which class plays audio?**

✓ MediaPlayer

---

**Which view displays video?**

✓ VideoView

---

**Which class records audio?**

✓ MediaRecorder

---

**Which permission accesses camera?**

✓ CAMERA

---

**Which permission records sound?**

✓ RECORD\_AUDIO

---

**Which view is used in games?**

✓ SurfaceView

---

## **Important Exam Question (15 Marks)**

**Explain Multimedia in Android. Discuss Image Handling, MediaPlayer, VideoView, MediaRecorder, Camera Access, Runtime Permissions, SurfaceView, advantages, disadvantages, and provide complete code examples.**



# 21. Android NDK (Native Development Kit) – Complete Detailed Notes

## ★ Important Theory Topic for Exams

Most Android apps are developed using:

Java  
or  
Kotlin

But sometimes developers need:

- High Performance
- Game Engines
- Image Processing
- Hardware Access
- Existing C/C++ Libraries

For these situations Android provides:

## Android NDK

---

### What is NDK?

#### Definition

NDK (Native Development Kit) is a set of tools that allows Android developers to write parts of their applications using C and C++.

---

#### Simple Words

NDK allows Android apps to use C/C++ code along with Java/Kotlin.

---

#### Real World Example

---

## PUBG Mobile

Java/Kotlin

↓

User Interface

↓

C/C++

↓

Game Engine

The game logic is mostly written in C++ for high performance.

---

## Why Use NDK?

Normally Android Apps:

Java

↓

Android Runtime

↓

Execution

---

Using NDK:

Java

↓

JNI

↓

C/C++

↓

Execution

This provides faster execution in some cases.

---

---

# Android SDK vs Android NDK

Very Important Exam Question

Android SDK	Android NDK
Uses Java/Kotlin	Uses C/C++
Easier Development	More Complex
Standard Apps	High Performance Apps
Android APIs	Native Libraries
Most Apps Use SDK	Specialized Apps Use NDK

---

## What is Native Code?

### Definition

Code written in:

C  
C++

and executed directly by the device processor.

---

### Example

Native Code:

```
int sum(int a,int b)
{
return a+b;
}
```

---

Java Code:

```
int sum(int a,int b)
{
return a+b;
}
```

---

Both perform same work.

But native code executes closer to hardware.

---

## NDK Architecture

Android App

↓

Java/Kotlin

↓

JNI

↓

Native Library (.so)

↓

C/C++

↓

Hardware

---

## Components of NDK

### 1. C/C++ Source Files

Example:

native-lib.cpp

---

### 2. JNI

Java Native Interface

Acts as bridge.

---

## 3. Native Libraries

Generated as:

```
libnative-lib.so
```

---

## 4. Android App

Uses native methods.

---

# What is JNI?

## Definition

JNI (Java Native Interface) is a framework that allows Java code to communicate with C/C++ code.

---

## Simple Words

JNI is a bridge between Java and C++.

---

# Real World Example

Java

↓

JNI

↓

C++

Without JNI:

Java and C++ cannot communicate.

---

---

# JNI Architecture

Java Code

↓

JNI

↓

Native Method

↓

C++ Code

↓

Result Returned

---

# Example Flow

Button Click

↓

Java Method

↓

JNI

↓

C++ Function

↓

Result

↓

TextView Updated

---

# Native Libraries

## Definition

Compiled C/C++ code is stored in:

`.so`

files.

---

## Example

`libgame.so`

`libopencv.so`

`libnative.so`

---

# What is .so File?

## Definition

Shared Object File

Equivalent of:

DLL

in Windows.

---

## Example

Windows

↓

`abc.dll`

Linux/Android

↓

`abc.so`

---

---

# Creating Native Method

## Java

```
public native String getMessage();
```

Keyword:

```
native
```

means method is implemented in C/C++.

---

## Load Native Library

```
static {

 System.loadLibrary(
 "native-lib");

}
```

---

## What Happens?

Android loads:

```
libnative-lib.so
```

into memory.

---

## Native C++ Implementation

### native-lib.cpp

```
#include <jni.h>
#include <string>

extern "C"
```

---



```
JNIEXPORT jstring JNICALL
Java_com_example_myapp_MainActivity_getMessage(
JNIEnv* env,
jobject obj) {
return env->NewStringUTF(
"Hello from C++");
}
```

---

## Output

Hello from C++

displayed in Android app.

---

## Complete Flow

Java Method

↓

getMessage()

↓

JNI

↓

C++ Function

↓

Hello from C++

↓

Returned to Java

---

## Passing Data Java → C++

## Java

```
public native int add(
int a,
int b);
```

---

## C++

```
JNIEXPORT jint JNICALL
Java_com_example_myapp_MainActivity_add(

JNIEnv* env,

jobject obj,

jint a,

jint b) {
return a+b;
}
```

---

## Java Call

```
int result=
add(10,20);
```

---

## Output

```
30
```

---

## Passing String Java → C++

### Java

```
public native String welcome(
String name);
```

---

## C++

```
JNIEXPORT jstring JNICALL
Java_com_example_myapp_MainActivity_welcome(
 JNIEnv* env,
 jobject obj,
 jstring name){
 return env->NewStringUTF(
 "Welcome");
}
```

---

## Output

Welcome

---

## Why Games Use NDK?

Games require:

- High FPS
  - Fast Graphics
  - Memory Optimization
- 

## Example

PUBG Mobile

User Input

↓

C++ Game Engine

↓

Graphics Rendering

↓

---

## OpenCV + NDK

OpenCV is image processing library.

Written in:

C++

Android uses NDK to integrate OpenCV.

---

## Real World Example

Face Detection App

Camera

↓

OpenCV

↓

NDK

↓

Face Detection

---

## NDK Applications

### Gaming

PUBG

Free Fire

Call of Duty

---

## **Image Processing**

OpenCV

Photo Editors

---

## **Audio Processing**

Spotify

Music Editors

---

## **Machine Learning**

TensorFlow Lite

AI Applications

---

## **Video Processing**

YouTube

Video Editors

---

## **Advantages of NDK**

### **1. High Performance**

Faster execution.

---

### **2. Reuse Existing Libraries**

Existing C/C++ code can be used.

---

### **3. Better Hardware Access**

Closer to CPU.

---

### **4. Gaming Support**

Ideal for game engines.

---

### **5. Image Processing**

Efficient multimedia processing.

---

## **Disadvantages of NDK**

### **1. Complex Development**

C++ is harder than Java.

---

### **2. Difficult Debugging**

Debugging native code is more difficult.

---

### **3. More Bugs**

Memory leaks may occur.

---

## 4. Increased APK Size

Native libraries increase app size.

---

## 5. Platform Dependency

Different processors may require different builds.

---

# Android ABI

## Definition

ABI = Application Binary Interface

Defines CPU architecture.

---

## Common ABIs

`armeabi-v7a`

`arm64-v8a`

`x86`

`x86_64`

---

## Why ABI Matters?

Different devices use different CPUs.

NDK generates separate .so files.

---

## Android App Using NDK

MainActivity

↓

JNI

↓

libnative.so

↓

C++ Code

↓

Hardware

---

## SDK + NDK Together

Most Android apps use both.

Android App

├─ Java/Kotlin (SDK)

└─ C/C++ (NDK)

---

## Real World Examples

### PUBG Mobile

Uses:

NDK

OpenGL

C++

---

### Free Fire

Uses:

C++

---



NDK

---

## OpenCV Face Detection

Uses:

Camera

↓

NDK

↓

OpenCV

---

## TensorFlow Lite

Uses:

AI Models

↓

Native Processing

↓

NDK

---

## SDK vs NDK Flow

### SDK

Java

↓

Android Runtime

↓

Output

---

# NDK

Java

↓

JNI

↓

C++

↓

Output

---

## Viva Questions

### What is NDK?

Android toolkit that allows use of C/C++ code.

---

### What does NDK stand for?

Native Development Kit.

---

### What is JNI?

Java Native Interface.

Bridge between Java and C++.

---

### Which languages are used in NDK?

C and C++.

---

### What is .so file?

Shared Object file.

Native library.

---

### **Why is NDK used?**

For performance-critical applications.

---

### **Can Android apps use both SDK and NDK?**

Yes.

Most large apps use both.

---

## **MCQs**

### **NDK stands for?**

✓ Native Development Kit

---

### **JNI stands for?**

✓ Java Native Interface

---

### **Native libraries use which extension?**

✓ .so

---

### **NDK primarily uses?**

✓ C/C++

---

---

**Which type of apps commonly use NDK?**

✓ Games

---

**JNI acts as?**

✓ Bridge between Java and C++

---

## **Important Exam Question (15 Marks)**

**Explain Android NDK (Native Development Kit). Discuss JNI, Native Libraries (.so files), C/C++ Integration, SDK vs NDK, advantages, disadvantages, architecture, and provide suitable code examples.**